République Tunisienne

Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique

École Supérieure Privée d'Ingénierie et de Technologie

TEK-UP

Rapport du Projet

Spécialité : Sécurité des Systèmes Informatiques et Réseaux

Réalisé par

Rabeb Jerbi

Chaima Romdhani

PatchMind

Plateforme DevSecOps intelligente pour la détection et la remédiation automatisée des vulnérabilités applicatives

Année Universitaire : 2025 – 2026

Liste des abréviations

API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
CVE	Common Vulnerabilities and Exposures
CVSS	Common Vulnerability Scoring System
CWE	Common Weakness Enumeration
DAST	Dynamic Application Security Testing
FAISS	Facebook AI Similarity Search
IA	Intelligence Artificielle
LLM	Large Language Model
MTTR	Mean Time To Remediate
NVD	National Vulnerability Database
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
SAST	Static Application Security Testing
SCA	Software Composition Analysis
TOTP	Time-based One-Time Password
ZAP	Zed Attack Proxy

Table des matières

Liste des abréviations **i**

Introduction Générale **1**

1 Contexte général du projet **3**
1.1 Introduction . . . 3
1.2 Présentation de l'établissement . . . 3
1.3 Contexte général et problématique . . . 4
1.4 Présentation de la solution proposée : PatchMind . . . 5
1.5 Objectifs du projet . . . 6
1.6 Méthodologie de gestion du projet : Scrum . . . 6
1.7 Organisation des releases et sprints . . . 7
1.8 Conclusion . . . 7

2 Fondements technologiques et étude comparative **8**
2.1 Introduction . . . 8
2.2 DevSecOps et sécurité applicative . . . 8
2.2.1 DevSecOps . . . 8
2.2.2 Les techniques d'analyse de sécurité . . . 9
2.3 Intelligence artificielle appliquée à la sécurité . . . 10
2.3.1 Large Language Models (LLM) . . . 10
2.3.2 Retrieval-Augmented Generation (RAG) . . . 10
2.3.3 FAISS et embeddings . . . 11
2.3.4 Consensus multi-LLM . . . 11
2.4 Standards de classification des vulnérabilités . . . 11
2.4.1 Common Weakness Enumeration (CWE) . . . 11
2.4.2 Common Vulnerabilities and Exposures (CVE) et NVD . . . 12
2.4.3 Common Vulnerability Scoring System (CVSS) . . . 12
2.5 Étude comparative des outils existants . . . 12
2.6 Limites des solutions actuelles . . . 13
2.7 Conclusion . . . 14

3 Analyse et conception de PatchMind **15**

3.1 Introduction . . . 15
3.2 Besoins fonctionnels et non fonctionnels . . . 15
3.2.1 Besoins fonctionnels . . . 15
3.2.2 Besoins non fonctionnels . . . 16
3.3 Architecture générale et pipeline . . . 18
3.3.1 Architecture logique de PatchMind . . . 18
3.3.2 Pipeline de traitement des vulnérabilités . . . 19
3.4 Modélisation UML . . . 20
3.4.1 Diagramme de cas d'utilisation . . . 20
3.4.2 Diagramme de classes . . . 21
3.5 Planification Scrum . . . 22
3.5.1 Product Backlog . . . 22
3.5.2 Organisation des sprints . . . 24
3.6 Conclusion . . . 24

4 Mise en place du pipeline DevSecOps de PatchMind 26
4.1 Introduction . . . 26
4.2 Sprint 1 — Authentification et gestion des utilisateurs . . . 26
4.2.1 Authentification . . . 27
4.2.2 Gestion des sessions . . . 28
4.2.3 Contrôle d'accès basé sur les rôles (RBAC) . . . 28
4.2.4 Double authentification (2FA) . . . 29
4.2.5 Validation des demandes d'accès . . . 30
4.2.6 Sécurisation de la plateforme . . . 30
4.2.7 Interfaces utilisateur . . . 31
4.3 Sprint 2 — Intégration des outils de sécurité . . . 32
4.3.1 Architecture du pipeline de scan . . . 33
4.3.2 Intégration de Semgrep . . . 34
4.3.3 Intégration de GitLeaks . . . 34
4.3.4 Intégration de Snyk . . . 34
4.3.5 Intégration d'OWASP ZAP . . . 35
4.3.6 Outils complémentaires optionnels . . . 35
4.3.7 Centralisation des résultats . . . 36
4.3.8 Exécution du pipeline d'analyse . . . 37
4.3.9 Résultats des analyses . . . 37
4.3.10 Automatisation du pipeline . . . 38
4.3.11 Difficultés rencontrées . . . 38
4.4 Conclusion . . . 39

5 Automatisation de la remédiation et supervision de la plateforme 40

5.1 Introduction 40
5.2 Sprint 3 — Remédiation intelligente et reporting 40
5.2.1 Génération intelligente des correctifs 40
5.2.2 Validation automatique des correctifs 42
5.2.3 Outils complémentaires 43
5.2.4 Intégration CI/CD 43
5.2.5 Génération des rapports PDF 44
5.2.6 Gestion de l'historique 45
5.2.7 Gestion collaborative des projets 46
5.2.8 Stockage des analyses 47
5.2.9 Consultation des résultats 47
5.3 Sprint 4 — Dashboard administrateur et tests 48
5.3.1 Dashboard administrateur 48
5.3.2 Statistiques et métriques 49
5.3.3 Journaux d'audit 50
5.3.4 Tests unitaires 51
5.3.5 Tests d'intégration 53
5.3.6 Tests de sécurité 53
5.3.7 Tests de performance 54
5.4 Difficultés rencontrées 54
5.4.1 Fiabilité des correctifs générés 54
5.4.2 Validation des correctifs 55
5.4.3 Performances du moteur IA 55
5.4.4 Cohérence entre reporting, historique et base de données 55
5.5 Résultats obtenus 55
5.5.1 Résultats de détection 55
5.5.2 Qualité des correctifs générés 56
5.5.3 Apport de PatchMind par rapport aux approches classiques 56
5.5.4 Performances de la plateforme 56
5.5.5 Réduction du MTTR 57
5.6 Conclusion 57

Conclusion Générale **58**

Table des figures

3.1 Architecture générale de la plateforme PatchMind 19
3.2 Pipeline automatisé de traitement des vulnérabilités dans PatchMind . . . 20
3.3 Diagramme de cas d'utilisation de PatchMind 21
3.4 Diagramme de classes de PatchMind 22

4.1 Page d'accueil de la plateforme PatchMind 27
4.2 Interface de connexion de PatchMind 28
4.3 Configuration de la double authentification (TOTP) 30
4.4 Tableau de bord utilisateur de PatchMind 31
4.5 Interface de gestion du profil utilisateur 32
4.6 Interface de gestion des projets utilisateur 32
4.7 Interface de soumission d'un fichier et sélection des scanners 33
4.8 Affichage centralisé des vulnérabilités détectées 36
4.9 Pipeline d'analyse en cours d'exécution 37
4.10 Affichage des vulnérabilités détectées et de leur statut de correction 38

5.1 Pipeline du moteur IA de génération et de sélection des correctifs 41
5.2 Visualisation du diff avant/après correction généré par PatchMind 42
5.3 Interface des fichiers corrigés avec les références CWE associées 42
5.4 Historique des exécutions du pipeline CI/CD GitHub Actions de PatchMind 43
5.5 Page de garde du rapport d'analyse généré par PatchMind 44
5.6 Contenu détaillé du rapport d'analyse — vulnérabilités et métriques 45
5.7 Interface de l'historique des analyses 46
5.8 Interface de gestion collaborative des projets PatchMind 47
5.9 Interface de suivi d'un projet et évolution des vulnérabilités 48
5.10 Dashboard administrateur de PatchMind 49
5.11 Interface de monitoring et supervision du système PatchMind 50
5.12 Journaux d'audit de PatchMind 51
5.13 Organisation de la suite de tests unitaires de PatchMind 52
5.14 Résultats des 500 tests unitaires Pytest de PatchMind 53

Liste des tableaux

1.1 Organisation des releases et sprints 7

2.1 Étude comparative des outils existants 13

3.1 Product Backlog du projet PatchMind 24
3.2 Organisation des releases et des sprints 24

4.1 Droits d'accès par rôle 29

5.1 Résultats des tests de performance 54
5.2 Comparaison entre l'approche classique et PatchMind 56

Introduction Générale

La transformation numérique accélérée des organisations a profondément modifié le paysage de la cybersécurité. La multiplication des applications web, la complexité croissante des bases de code ainsi que la généralisation des architectures distribuées ont considérablement élargi la surface d'attaque exposée aux menaces informatiques. Dans ce contexte, les vulnérabilités logicielles représentent un risque critique, d'autant plus que le délai moyen de remédiation — ou *Mean Time To Remediate* (MTTR) — demeure élevé, atteignant parfois plusieurs semaines dans les environnements complexes, laissant ainsi les systèmes exposés pendant des périodes prolongées.

Le paradigme DevSecOps s'est imposé comme une réponse à cette problématique en intégrant la sécurité au cœur du cycle de développement logiciel. Cependant, bien que les outils d'analyse existants tels que Semgrep, OWASP ZAP ou Snyk permettent de détecter efficacement les vulnérabilités, ils n'offrent généralement pas de mécanismes avancés de remédiation automatisée. La correction des failles reste alors un processus souvent manuel, complexe et chronophage, nécessitant une expertise approfondie en cybersécurité.

Par ailleurs, l'émergence des grands modèles de langage (*Large Language Models — LLM*) ouvre de nouvelles perspectives dans le domaine de la sécurité applicative. Grâce à leurs capacités de compréhension et de génération de code, ces modèles permettent d'envisager la génération automatique de correctifs adaptés au contexte des vulnérabilités détectées. Cependant, utilisés isolément, ils présentent des limites significatives en termes de précision contextuelle et de fiabilité, notamment en raison du risque d'hallucinations et de l'absence de connaissance spécifique au domaine de la sécurité.

C'est pourquoi PatchMind associe les modèles de langage à une architecture RAG (*Retrieval-Augmented Generation*), permettant d'ancrer les réponses dans une base de connaissances spécialisée, ainsi qu'à un mécanisme de consensus multi-LLM afin d'améliorer significativement la pertinence et la robustesse des correctifs générés. Les recherches sémantiques sont assurées par FAISS afin de retrouver les contextes de vulnérabilités les plus pertinents. Le système exploite notamment Gemini, Groq API et Ollama afin de générer plusieurs propositions de correctifs et sélectionner les réponses les plus fiables.

C'est dans ce contexte qu'a été développé PatchMind, une plateforme DevSecOps

intelligente dédiée à la détection, à l'analyse et à la remédiation automatique des vulnérabilités applicatives. La plateforme centralise les résultats de plusieurs outils d'analyse de sécurité tels que Semgrep, GitLeaks, OWASP ZAP et Snyk/OSV, automatise la génération de correctifs à l'aide de modèles d'intelligence artificielle et applique plusieurs mécanismes de validation aux correctifs générés afin d'en garantir la qualité et la cohérence.

PatchMind vise ainsi à réduire significativement le MTTR tout en améliorant l'automatisation, la centralisation et l'efficacité des processus de sécurisation des applications.

Afin de présenter les différentes étapes de réalisation du projet, ce rapport est structuré en cinq chapitres.

Le premier chapitre présente le contexte général du projet, la problématique étudiée ainsi que la méthodologie de gestion adoptée.

Le deuxième chapitre expose les technologies DevSecOps utilisées ainsi que l'étude comparative des solutions de sécurité applicative existantes, mettant en évidence leurs limites et justifiant ainsi les choix architecturaux de PatchMind.

Le troisième chapitre est consacré à l'analyse des besoins et à la conception de la plateforme PatchMind, notamment l'architecture générale, le pipeline de traitement des vulnérabilités, la modélisation UML ainsi que la conception de la base de données.

Le quatrième chapitre présente l'implémentation des mécanismes de sécurité et du pipeline d'analyse, incluant l'authentification, la gestion des utilisateurs ainsi que l'intégration des outils de sécurité.

Le cinquième chapitre détaille l'implémentation du moteur intelligent de remédiation et de supervision, les mécanismes de génération et de validation des correctifs, les tests réalisés ainsi que les résultats obtenus et les perspectives d'amélioration futures.

Chapitre 1

Contexte général du projet

1.1 Introduction

Ce chapitre présente le cadre général dans lequel s'inscrit le projet PatchMind. Il aborde le contexte académique du projet, la problématique étudiée ainsi que les objectifs poursuivis. La solution proposée est ensuite présentée de manière globale avant de détailler la méthodologie de gestion adoptée et l'organisation des différentes phases de développement.

1.2 Présentation de l'établissement

TEK-UP est une école supérieure privée d'ingénierie et de technologie proposant plusieurs formations spécialisées dans les domaines de l'informatique, des télécommunications et des systèmes d'information. Parmi ces formations figure la spécialité Sécurité des Systèmes Informatiques et Réseaux, orientée vers les problématiques liées à la cybersécurité, à la sécurisation des infrastructures, à la protection des données ainsi qu'à la sécurité applicative.

Cette spécialité met notamment l'accent sur les pratiques modernes de sécurisation des systèmes d'information, en particulier l'intégration de la sécurité dans le cycle de développement logiciel à travers les approches DevSecOps et l'utilisation d'outils d'analyse automatisée.

Dans le cadre de cette formation, les étudiants sont amenés à concevoir et réaliser des projets techniques répondant à des problématiques réelles du domaine de la cybersécurité. Ces projets permettent de mettre en pratique les connaissances théoriques acquises tout en développant des compétences en développement logiciel, administration système, analyse

de sécurité et automatisation.

TEK-UP encourage également les projets orientés innovation et cybersécurité appliquée, permettant aux étudiants de développer des solutions répondant aux enjeux actuels de la sécurité des systèmes d'information.

Le projet PatchMind a ainsi été développé dans ce contexte académique afin de répondre aux problématiques liées à l'automatisation de la sécurité applicative et à l'intégration de mécanismes intelligents de remédiation des vulnérabilités.

1.3 Contexte général et problématique

Avec l'accélération de la transformation numérique, les organisations s'appuient de plus en plus sur des applications web, des services distribués et des architectures modernes fortement connectées. Cette évolution a considérablement augmenté la surface d'attaque des systèmes d'information et multiplié les risques liés aux vulnérabilités logicielles.

Dans ce contexte, les vulnérabilités applicatives représentent aujourd'hui l'une des principales causes des incidents de cybersécurité. Les erreurs de développement, les mauvaises configurations, les dépendances vulnérables ou encore l'exposition accidentelle de secrets peuvent compromettre la sécurité des applications et des données sensibles.

Afin de répondre à ces menaces, les approches DevSecOps intègrent la sécurité au cœur du cycle de développement logiciel à travers l'utilisation d'outils d'analyse automatisée. Plusieurs solutions permettent aujourd'hui de détecter efficacement les vulnérabilités applicatives grâce aux analyses statiques, dynamiques ou encore à l'analyse des dépendances.

Cependant, malgré l'efficacité des outils de détection existants, la remédiation des vulnérabilités demeure un processus largement manuel. Une fois une faille détectée, les développeurs doivent analyser les résultats, comprendre la vulnérabilité, rechercher une solution adaptée puis vérifier eux-mêmes la validité du correctif avant son intégration. Cette approche présente plusieurs limites :

— un délai de remédiation élevé ($MTTR$) ;
— une forte dépendance à l'expertise humaine ;
— des risques d'erreurs ou de régressions fonctionnelles ;
— une absence de centralisation des résultats des différents outils ;
— un manque de mécanismes intelligents de génération automatique de correctifs ;
— l'absence de validation automatisée des remédiations proposées.

Par ailleurs, l'émergence récente des grands modèles de langage (LLM) ouvre de nou-

velles perspectives dans le domaine de la sécurité applicative. Grâce à leurs capacités de compréhension et de génération de code, ces modèles permettent d'envisager l'automatisation partielle de la correction des vulnérabilités.

Cependant, utilisés isolément, ils présentent certaines limites significatives, notamment les hallucinations, le manque de précision contextuelle ainsi que l'absence de connaissances spécialisées en sécurité applicative.

Dans ce contexte, la question centrale à laquelle répond ce projet est la suivante :

Comment automatiser l'intégralité du cycle de traitement des vulnérabilités applicatives — de leur détection jusqu'à la validation des correctifs — au sein d'une plateforme DevSecOps intelligente et centralisée ?

1.4 Présentation de la solution proposée : PatchMind

Afin de répondre aux limites identifiées, le projet PatchMind propose une plateforme DevSecOps intelligente dédiée à la détection, à l'analyse et à la remédiation automatisée des vulnérabilités applicatives.

PatchMind permet aux utilisateurs de soumettre un projet, un fichier source ou un dépôt GitHub afin de lancer une analyse de sécurité automatisée et centralisée. La plateforme intègre quatre outils d'analyse spécialisés couvrant les principaux aspects de la sécurité applicative : Semgrep pour l'analyse statique (SAST), GitLeaks pour la détection de secrets exposés, Snyk pour l'analyse des dépendances (SCA) et OWASP ZAP pour l'analyse dynamique (DAST).

En complément de la détection, PatchMind exploite des mécanismes d'intelligence artificielle afin de générer automatiquement des propositions de correctifs adaptées au contexte des vulnérabilités détectées. La plateforme repose sur une architecture RAG permettant d'enrichir les réponses des modèles IA à partir d'une base de connaissances spécialisée en sécurité applicative. Les recherches sémantiques sont assurées par FAISS afin de retrouver les contextes de vulnérabilités les plus pertinents.

PatchMind utilise également un mécanisme de consensus multi-LLM s'appuyant sur plusieurs modèles d'intelligence artificielle — Groq, Gemini et Ollama — permettant de comparer plusieurs propositions de correctifs et de sélectionner la réponse la plus fiable.

Enfin, la plateforme intègre plusieurs mécanismes de sécurité et de supervision, notamment l'authentification sécurisée avec double authentification (2FA), le contrôle d'accès basé sur les rôles (RBAC), la journalisation des événements de sécurité, le suivi des analyses ainsi que la génération de rapports et de statistiques via un dashboard administrateur centralisé.

L'objectif principal de PatchMind est de réduire significativement le délai moyen de remédiation des vulnérabilités tout en améliorant l'automatisation, la centralisation et l'efficacité des processus de sécurisation applicative.

1.5 Objectifs du projet

Sur la base de la solution présentée, les principaux objectifs poursuivis par PatchMind sont les suivants :

- Automatiser les analyses de sécurité applicative à travers l'intégration de plusieurs scanners spécialisés au sein d'un pipeline unifié ;
- Centraliser les résultats des analyses au sein d'une plateforme unique et accessible ;
- Générer automatiquement des correctifs intelligents à l'aide d'une architecture RAG et d'un mécanisme de consensus multi-LLM ;
- Valider les correctifs proposés grâce à plusieurs couches de vérification automatisée ;
- Renforcer la sécurité de la plateforme à travers des mécanismes d'authentification 2FA et de contrôle d'accès RBAC ;
- Assurer la supervision et la traçabilité des analyses grâce à des mécanismes de monitoring et de journalisation ;
- Améliorer l'efficacité des équipes de développement dans le traitement des vulnérabilités ;
- Réduire significativement le *(MTTR)*.

1.6 Méthodologie de gestion du projet : Scrum

La gestion du projet PatchMind repose sur la méthode agile Scrum. Cette approche organise le développement en cycles courts et itératifs appelés sprints, chacun correspondant à un ensemble de fonctionnalités défini et livrable à l'issue d'une période déterminée.

Le choix de la méthode Scrum s'explique par la nature fortement modulaire et évolutive du projet PatchMind. En effet, l'intégration successive de composants hétérogènes — outils d'analyse de sécurité, moteur RAG, mécanisme de consensus multi-LLM et interfaces de supervision — rendait indispensable une approche permettant de valider chaque composant indépendamment avant intégration, d'intégrer progressivement les fonctionnalités et d'adapter le développement aux résultats obtenus à chaque itération.

Cette méthodologie favorise une progression incrémentale du produit, une meilleure organisation des tâches, une visibilité claire sur l'avancement du projet ainsi qu'une capacité d'adaptation continue aux évolutions des besoins.

1.7 Organisation des releases et sprints

Le développement de PatchMind a été organisé selon une approche incrémentale basée sur deux releases principales, elles-mêmes divisées en quatre sprints. Cette organisation a permis de structurer progressivement le développement de la plateforme tout en validant régulièrement les fonctionnalités réalisées.

La première release a principalement porté sur la mise en place des fonctionnalités fondamentales de la plateforme, notamment l'authentification, la gestion des utilisateurs, les mécanismes de sécurité ainsi que l'intégration des outils d'analyse et du pipeline automatisé de détection des vulnérabilités.

La deuxième release a été consacrée aux fonctionnalités avancées de remédiation intelligente, incluant la génération automatique des correctifs, les mécanismes de validation, les fonctionnalités de supervision, le reporting ainsi que les tests et validations finales du système.

Le tableau suivant présente le détail de l'organisation du projet :

TABLE 1.1 – Organisation des releases et sprints

Release	**Sprint**	**Objectif principal**
Release 1	Sprint 1	Authentification, gestion des rôles (RBAC), double authentification (2FA)
Release 1	Sprint 2	Intégration de Semgrep, GitLeaks, Snyk, OWASP ZAP et mise en place du pipeline d'analyse
Release 2	Sprint 3	Architecture RAG, FAISS, consensus multi-LLM et génération automatique de correctifs
Release 2	Sprint 4	Dashboard administrateur, reporting, tests et validation finale

1.8 Conclusion

Ce chapitre a présenté le contexte général du projet PatchMind ainsi que le cadre académique dans lequel il a été réalisé. Il a exposé la problématique abordée, la solution proposée, les objectifs poursuivis ainsi que la méthodologie Scrum adoptée pour organiser le développement de la plateforme.

Le chapitre suivant présente les principales technologies DevSecOps, les concepts d'intelligence artificielle exploités ainsi que les solutions de sécurité applicative existantes, mettant en évidence leurs limites et justifiant les choix architecturaux de PatchMind.

Chapitre 2

Fondements technologiques et étude comparative

2.1 Introduction

Ce chapitre présente les principales technologies et concepts sur lesquels repose le projet PatchMind. Il aborde les pratiques DevSecOps et les techniques d'analyse de sécurité applicative, les apports de l'intelligence artificielle dans le domaine de la remédiation des vulnérabilités, les standards de classification utilisés dans l'industrie, ainsi qu'une étude comparative des outils existants et de leurs limites.

2.2 DevSecOps et sécurité applicative

2.2.1 DevSecOps

Le DevSecOps est une évolution du paradigme DevOps intégrant la sécurité comme une responsabilité partagée entre les équipes de développement, d'exploitation et de sécurité tout au long du cycle de vie logiciel. Contrairement aux approches traditionnelles où les contrôles de sécurité sont réalisés en fin de cycle, le DevSecOps préconise une détection et une correction précoces des vulnérabilités, dès les premières phases de développement.

Cette philosophie repose sur l'automatisation des tests de sécurité, leur intégration dans les pipelines CI/CD ainsi que la mise en place d'un retour d'information rapide à destination des développeurs. Elle vise à réduire le coût et le délai de remédiation des vulnérabilités tout en renforçant la qualité et la sécurité des applications livrées.

2.2.2 Les techniques d'analyse de sécurité

La sécurité applicative s'appuie sur plusieurs techniques d'analyse complémentaires, chacune ciblant un aspect différent de la surface de vulnérabilité d'une application.

Static Application Security Testing (SAST)

Le SAST désigne les techniques d'analyse statique du code source permettant d'identifier des vulnérabilités sans exécuter l'application. Ces outils examinent le code source, les fichiers de configuration et les dépendances afin de détecter des failles telles que les injections SQL (CWE-89), les vulnérabilités XSS (CWE-79) ou les erreurs d'authentification (CWE-287).

L'avantage principal du SAST réside dans sa capacité à intervenir très tôt dans le cycle de développement. Cependant, ces outils peuvent générer des faux positifs nécessitant une validation manuelle.

Dynamic Application Security Testing (DAST)

Le DAST consiste à analyser une application en cours d'exécution en simulant des attaques réelles afin d'identifier des vulnérabilités exploitables dynamiquement. Contrairement au SAST, il permet de détecter des failles difficilement visibles à l'analyse statique, telles que les problèmes de gestion des sessions ou les erreurs de configuration d'environnement.

Il nécessite en revanche une application fonctionnelle et déployée, ce qui le rend plus coûteux en temps d'exécution.

Software Composition Analysis (SCA)

Le SCA permet d'identifier les vulnérabilités présentes dans les bibliothèques et dépendances tierces utilisées par une application. Avec la généralisation des composants open source, les risques liés à la chaîne d'approvisionnement logicielle (*Supply Chain Attack*) sont devenus une préoccupation majeure.

Les outils SCA analysent les fichiers de dépendances, détectent les versions vulnérables et les référencent par rapport aux bases de données de vulnérabilités connues telles que le NVD.

Secret Scanning

Le Secret Scanning consiste à détecter les informations sensibles exposées accidentellement dans le code source ou les historiques de dépôts Git. Ces informations peuvent inclure des clés API, des mots de passe, des jetons d'authentification ou des certificats.

Leur divulgation représente un risque critique pouvant compromettre l'ensemble de l'infrastructure d'une organisation.

Outils d'analyse complémentaires

En complément des quatre techniques principales, PatchMind intègre plusieurs outils d'analyse spécialisés selon les technologies et langages analysés, notamment Bandit et pylint pour le code Python, pip-audit pour les dépendances Python ainsi que npm-audit pour les dépendances JavaScript. Ces outils permettent d'affiner l'analyse en fonction du contexte technologique du projet soumis.

2.3 Intelligence artificielle appliquée à la sécurité

2.3.1 Large Language Models (LLM)

Les Large Language Models (LLM) sont des modèles d'intelligence artificielle entraînés sur de très grands volumes de données textuelles et de code source. Grâce aux techniques d'apprentissage profond, ces modèles sont capables de comprendre le langage naturel, d'analyser du code source et de générer des propositions de correction contextuellement adaptées.

Dans le domaine de la cybersécurité, ils offrent des perspectives prometteuses pour l'automatisation de l'analyse des vulnérabilités et la génération de correctifs.

Cependant, les LLM présentent des limites importantes : ils peuvent produire des réponses incorrectes ou incomplètes, générer des hallucinations, ou proposer des correctifs non adaptés au contexte réel de l'application analysée. Ces limitations rendent nécessaire l'utilisation de mécanismes complémentaires permettant d'améliorer la pertinence et la fiabilité des réponses générées.

2.3.2 Retrieval-Augmented Generation (RAG)

Le Retrieval-Augmented Generation (RAG) est une architecture combinant recherche d'informations et génération de contenu par intelligence artificielle. Elle consiste à récu-

pérer des informations pertinentes depuis une base de connaissances spécialisée avant de les transmettre au modèle de langage, afin d'enrichir le contexte disponible et d'améliorer la qualité des réponses générées.

Dans le cadre de PatchMind, cette approche permet de fournir aux modèles IA des exemples de vulnérabilités similaires et de correctifs validés, améliorant ainsi significativement la précision des correctifs proposés.

2.3.3 FAISS et embeddings

FAISS (*Facebook AI Similarity Search*) est une bibliothèque spécialisée dans la recherche de similarité vectorielle à grande échelle. Elle permet d'indexer des représentations vectorielles (*embeddings*) et d'effectuer des recherches rapides d'éléments similaires au sein de grandes bases de données.

Les embeddings correspondent à des représentations vectorielles générées à partir de textes, de descriptions de vulnérabilités ou de correctifs, permettant de mesurer la similarité sémantique entre différents éléments.

Dans le cadre de l'architecture RAG de PatchMind, FAISS est utilisé pour indexer des exemples de vulnérabilités et de correctifs, permettant ainsi une recherche efficace de cas similaires lors de la génération des correctifs.

2.3.4 Consensus multi-LLM

Afin de pallier les limites inhérentes à chaque modèle de langage pris individuellement, PatchMind adopte une approche de consensus multi-LLM consistant à interroger indépendamment plusieurs modèles pour une même vulnérabilité, puis à comparer les propositions obtenues selon des critères de similarité textuelle, de cohérence syntaxique et de score de confiance. Le correctif présentant le meilleur consensus inter-modèles est alors sélectionné, réduisant ainsi les risques d'erreurs et améliorant la fiabilité globale de la remédiation.

2.4 Standards de classification des vulnérabilités

2.4.1 Common Weakness Enumeration (CWE)

Le CWE est une classification standardisée des faiblesses logicielles et des mauvaises pratiques de développement. Chaque type de vulnérabilité est associé à un identifiant

CWE permettant de le catégoriser précisément, d'en comprendre l'origine et les impacts potentiels, et d'orienter les recommandations de correction.

Cette classification est utilisée par PatchMind pour structurer et présenter les vulnérabilités détectées de manière cohérente et exploitable.

2.4.2 Common Vulnerabilities and Exposures (CVE) et NVD

Le CVE est une base de référence publique recensant les vulnérabilités connues affectant les logiciels et systèmes informatiques. Chaque vulnérabilité reçoit un identifiant unique facilitant le suivi, le partage d'informations et la corrélation entre différents outils de sécurité.

Le National Vulnerability Database (NVD), maintenu par le NIST, enrichit ces données en fournissant les scores CVSS, les niveaux de sévérité et les références techniques associées à chaque vulnérabilité référencée.

2.4.3 Common Vulnerability Scoring System (CVSS)

Le CVSS est un système de notation standardisé permettant d'évaluer la gravité des vulnérabilités selon une échelle allant de faible à critique. Ces scores permettent aux équipes de sécurité de prioriser les corrections en fonction de l'impact potentiel de chaque faille, facilitant ainsi une gestion efficace des risques applicatifs.

2.5 Étude comparative des outils existants

Afin de mieux situer PatchMind par rapport aux solutions existantes, une étude comparative des principaux outils de sécurité applicative a été réalisée.

TABLE 2.1 – Étude comparative des outils existants

Outil	**Type**	**Points forts**	**Limites**
Semgrep	SAST	Rapide, règles personnalisables, multi-langages	Pas de remédiation automatique
GitLeaks	Secret Scanning	Détection efficace dans les dépôts Git	Aucune correction proposée
Snyk / OSV	SCA	Analyse des dépendances, surveillance continue	Orienté dépendances uniquement
OWASP ZAP	DAST	Tests dynamiques complets, open source	Pas de remédiation intelligente
PatchMind	Plateforme DevSecOps intelligente	Centralisation, génération automatique de correctifs, validation, consensus multi-LLM, supervision	Solution académique en évolution ; couverture linguistique en cours d'extension ; dépendance à la disponibilité des API LLM externes

L'analyse de ces outils révèle une limite commune et fondamentale : aucun ne propose de mécanisme automatisé complet de génération et de validation des correctifs. Ils se concentrent principalement sur la détection, laissant la remédiation largement à la charge des développeurs.

2.6 Limites des solutions actuelles

Malgré leurs performances dans la détection des vulnérabilités, les outils étudiés présentent plusieurs limites empêchant une automatisation complète du processus de remédiation.

Tout d'abord, la majorité des outils se concentrent uniquement sur la détection des vulnérabilités sans proposer de mécanismes avancés de remédiation automatisée. Les développeurs doivent analyser manuellement les résultats et concevoir eux-mêmes les correctifs nécessaires, ce qui requiert une expertise approfondie en cybersécurité que tous ne possèdent pas nécessairement.

De plus, ces outils sont généralement utilisés de manière isolée, produisant des résultats hétérogènes et difficiles à centraliser. Cette fragmentation complique la gestion globale des vulnérabilités et allonge considérablement les délais de traitement.

Les mécanismes de validation automatique des correctifs demeurent également absents des solutions actuelles, augmentant les risques d'erreurs et de régressions lors de

l'application des correctifs.

Les approches basées uniquement sur les modèles d'intelligence artificielle présentent également des risques d'hallucinations et de génération de correctifs non adaptés au contexte réel des applications analysées.

Par ailleurs, les solutions les plus complètes du marché sont souvent propriétaires et onéreuses, limitant leur accessibilité aux grandes organisations disposant de ressources importantes. Cette réalité exclut de fait les petites équipes et les projets open source d'une protection avancée et centralisée.

Enfin, l'absence de centralisation et de reporting structuré rend difficile le suivi de la posture de sécurité dans le temps, notamment pour les équipes de gouvernance et les auditeurs externes.

2.7 Conclusion

Ce chapitre a présenté les principaux fondements technologiques et concepts sur lesquels repose le projet PatchMind. Les techniques d'analyse de sécurité applicative — SAST, DAST, SCA et Secret Scanning — constituent le socle de détection de la plateforme, tandis que l'architecture RAG et le mécanisme de consensus multi-LLM en forment le moteur de remédiation intelligente.

L'étude comparative des outils existants a mis en évidence leurs limites communes, notamment l'absence de remédiation automatisée, de validation intelligente des correctifs et de centralisation accessible, justifiant ainsi le développement d'une solution intégrée telle que PatchMind.

Le chapitre suivant est consacré à l'analyse des besoins et à la conception de la plateforme PatchMind, détaillant l'architecture générale et le pipeline DevSecOps intelligent mis en place.

Chapitre 3

Analyse et conception de PatchMind

3.1 Introduction

Après avoir présenté les fondements technologiques et l'étude comparative des solutions existantes, ce chapitre est consacré à l'analyse des besoins et à la conception de la plateforme PatchMind.

Il présente tout d'abord les besoins fonctionnels et non fonctionnels identifiés lors de l'étude du projet, puis décrit l'architecture générale de la solution ainsi que le pipeline de traitement des vulnérabilités. Enfin, les diagrammes UML et la planification Scrum sont présentés afin de modéliser les composants principaux de la plateforme.

3.2 Besoins fonctionnels et non fonctionnels

Sur la base de la solution présentée au chapitre précédent, les besoins fonctionnels et non fonctionnels de PatchMind ont été identifiés comme suit.

3.2.1 Besoins fonctionnels

Les besoins fonctionnels représentent l'ensemble des fonctionnalités que la plateforme PatchMind doit fournir afin de répondre aux objectifs du projet. Ils ont été identifiés à partir de l'analyse de la problématique et des cas d'utilisation de la plateforme.

Authentification et gestion des accès

La plateforme doit permettre aux utilisateurs de s'authentifier de manière sécurisée. Elle intègre un système de double authentification (*2FA*) ainsi qu'un contrôle d'accès basé sur les rôles (*RBAC*) permettant de distinguer les droits des utilisateurs standards et des administrateurs. Un mécanisme de demande d'accès est également prévu pour les nouveaux utilisateurs.

Analyse de sécurité

PatchMind doit permettre la soumission de fichiers, de projets ou de dépôts GitHub afin de déclencher une analyse de sécurité automatisée. Les outils d'analyse sont exécutés de manière coordonnée et leurs résultats sont centralisés au sein d'une interface unifiée.

Chaque vulnérabilité détectée doit être associée à des informations exploitables telles que le type de faille, le fichier concerné, le niveau de sévérité, l'outil ayant détecté la vulnérabilité ainsi que les références CWE ou CVE lorsqu'elles sont disponibles.

Génération et validation des correctifs

La plateforme doit générer automatiquement des propositions de correctifs à l'aide de modèles d'intelligence artificielle. Ces correctifs sont évalués par un mécanisme de consensus multi-LLM puis soumis à plusieurs couches de validation avant d'être présentés à l'utilisateur.

Reporting et supervision

PatchMind doit permettre la génération automatique de rapports d'analyse au format PDF, la consultation de l'historique des analyses ainsi que l'accès à un tableau de bord administrateur présentant les statistiques et métriques globales de la plateforme.

Les journaux d'audit doivent également être disponibles afin d'assurer la traçabilité des opérations importantes.

3.2.2 Besoins non fonctionnels

Les besoins non fonctionnels décrivent les contraintes techniques et qualitatives auxquelles la plateforme doit répondre indépendamment des fonctionnalités métier.

Sécurité

La plateforme doit garantir un haut niveau de sécurité à travers la sécurisation des sessions utilisateur, le chiffrement des données sensibles, le contrôle d'accès basé sur les rôles et la journalisation des actions critiques. La double authentification constitue une exigence importante pour renforcer la sécurité de l'accès à la plateforme.

Performance

PatchMind doit être capable d'exécuter les analyses de sécurité dans des délais raisonnables tout en assurant une bonne réactivité de l'interface utilisateur. L'exécution des scanners doit être optimisée afin de minimiser le temps de traitement global de chaque analyse.

Scalabilité

L'architecture de la plateforme doit être suffisamment modulaire pour permettre l'intégration future de nouveaux outils d'analyse, de nouveaux modèles d'intelligence artificielle ainsi que la gestion de volumes d'analyses plus importants sans remise en cause de l'architecture existante.

Maintenabilité

Le code source de la plateforme doit respecter des standards de développement clairs facilitant les mises à jour, l'ajout de nouvelles fonctionnalités et la correction des anomalies. La persistance des données est assurée par une base de données SQLite gérée via l'ORM SQLAlchemy, garantissant une intégration native avec le backend Flask et facilitant les évolutions futures du modèle de données.

Ergonomie

L'interface utilisateur doit être simple, intuitive et adaptée aux besoins des développeurs comme des administrateurs, permettant une prise en main rapide sans nécessiter de formation préalable.

3.3 Architecture générale et pipeline

3.3.1 Architecture logique de PatchMind

PatchMind repose sur une architecture modulaire organisée en cinq couches distinctes, permettant de séparer clairement les responsabilités entre les différents composants de la plateforme. Cette séparation facilite la maintenance, l'évolution et l'intégration future de nouveaux modules.

L'architecture logique de PatchMind est composée des éléments suivants :

— **Interface utilisateur :** permet aux utilisateurs et aux administrateurs d'interagir avec la plateforme, de soumettre des projets, de consulter les résultats et de télécharger les rapports.

— **Backend Flask :** assure le traitement des requêtes, la gestion des sessions, le contrôle d'accès, la communication avec les modules internes et l'orchestration générale de la plateforme. Il est développé en Python à l'aide du framework Flask.

— **Moteur d'analyse de sécurité :** orchestre l'exécution coordonnée des différents outils de scan —Semgrep, GitLeaks, Snyk/OSV, OWASP ZAP ainsi que plusieurs outils optionnels tels que Bandit, pip-audit, pylint et npm-audit — et centralise leurs résultats au sein d'un format unifié.

— **Moteur IA :** exploite l'architecture RAG, FAISS et les modèles LLM afin de générer des propositions de correctifs adaptées aux vulnérabilités détectées.

— **Moteur de validation :** vérifie les correctifs générés à travers plusieurs mécanismes : validation syntaxique, re-scan de sécurité, détection de régressions et calcul d'un score de confiance.

— **Base de données :** assure la persistance des utilisateurs, projets, analyses, vulnérabilités, correctifs, métriques et journaux d'audit. Elle repose sur SQLite, gérée via l'ORM SQLAlchemy.

— **Module de reporting :** permet la génération de rapports PDF et la consultation de l'historique des analyses via un tableau de bord administrateur centralisé.

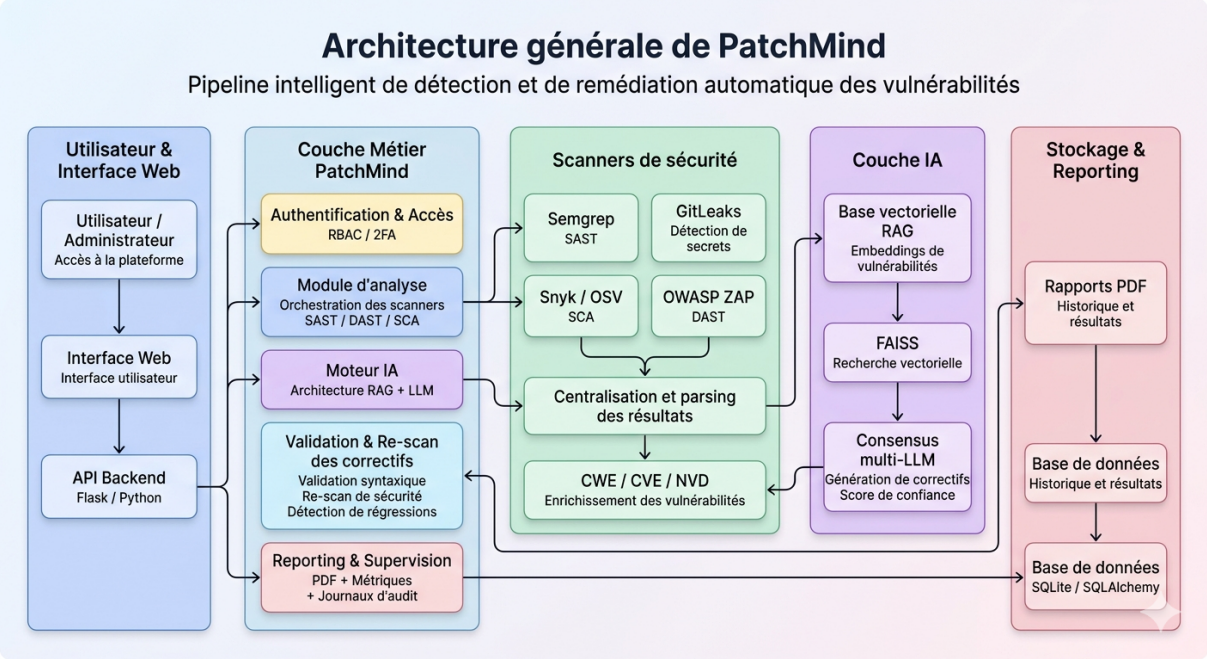


FIGURE 3.1 – Architecture générale de la plateforme PatchMind

3.3.2 Pipeline de traitement des vulnérabilités

Le fonctionnement de PatchMind repose sur un pipeline automatisé en neuf étapes permettant de traiter les vulnérabilités applicatives de manière progressive et structurée. Ce pipeline garantit que chaque vulnérabilité détectée est analysée, corrigée et validée avant d'être présentée à l'utilisateur.

Le pipeline suit les étapes suivantes :

1. Soumission du projet, du fichier ou du dépôt GitHub par l'utilisateur ;
2. Exécution coordonnée des scanners de sécurité ;
3. Centralisation et parsing des résultats bruts produits par chaque outil ;
4. Enrichissement des vulnérabilités détectées via les bases CWE, CVE et NVD ;
5. Recherche de contextes similaires dans la base de connaissances via l'architecture RAG et FAISS ;
6. Génération de plusieurs propositions de correctifs par les modèles LLM ;
7. Évaluation et sélection du meilleur correctif via le mécanisme de consensus multi-LLM ;
8. Validation syntaxique, re-scan de sécurité et détection de régressions ;
9. Génération du rapport PDF et stockage des résultats en base de données.

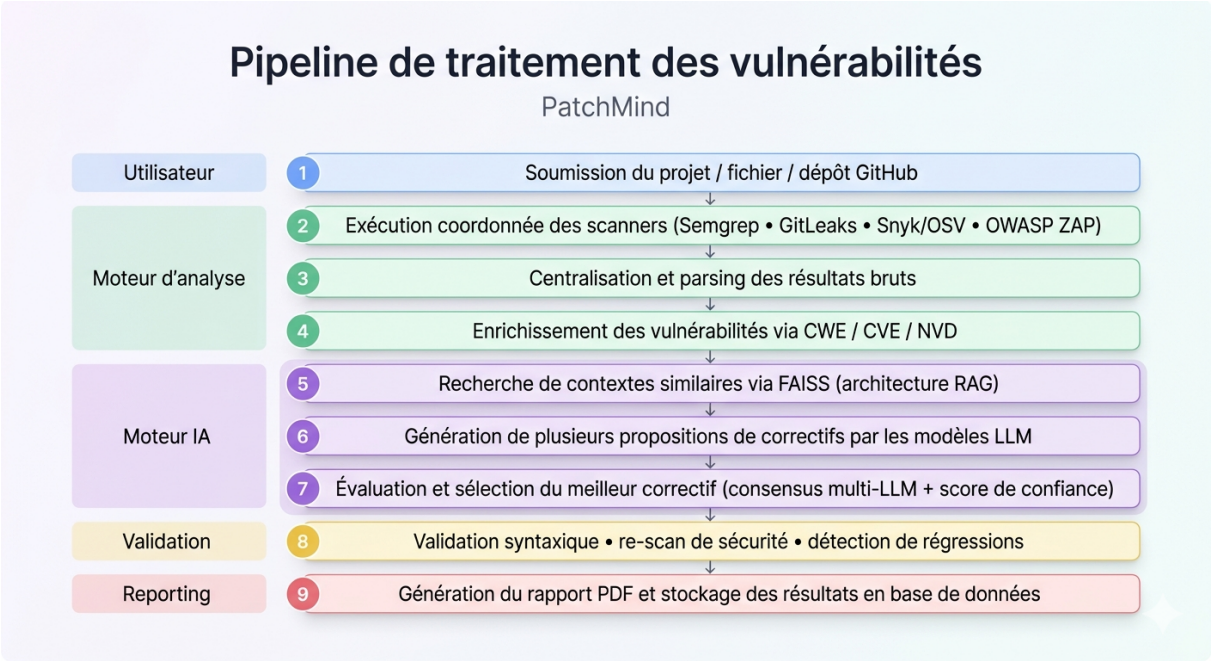


FIGURE 3.2 – Pipeline automatisé de traitement des vulnérabilités dans PatchMind

3.4 Modélisation UML

Afin de modéliser les interactions et la structure interne de la plateforme, plusieurs diagrammes UML ont été réalisés. Ces diagrammes permettent de représenter les acteurs, les fonctionnalités principales ainsi que les entités manipulées par le système.

3.4.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation représente les principales interactions entre les acteurs et la plateforme PatchMind. L'authentification constitue un prérequis global à l'ensemble des fonctionnalités de la plateforme — toute action nécessite que l'utilisateur soit préalablement connecté via le mécanisme 2FA.

Trois acteurs principaux ont été identifiés :

— **Utilisateur :** il peut s'authentifier, soumettre un projet, lancer une analyse, consulter les vulnérabilités détectées, visualiser les correctifs proposés, télécharger le fichier corrigé, télécharger les rapports PDF et consulter l'historique de ses analyses.
— **Administrateur :** il hérite des droits de l'utilisateur et dispose de fonctionnalités supplémentaires telles que la gestion des comptes utilisateurs, la consultation des journaux d'audit et la supervision globale de la plateforme.
— **Système IA externe :** il représente les modèles de langage sollicités automatiquement lors de la génération et de la validation des correctifs.

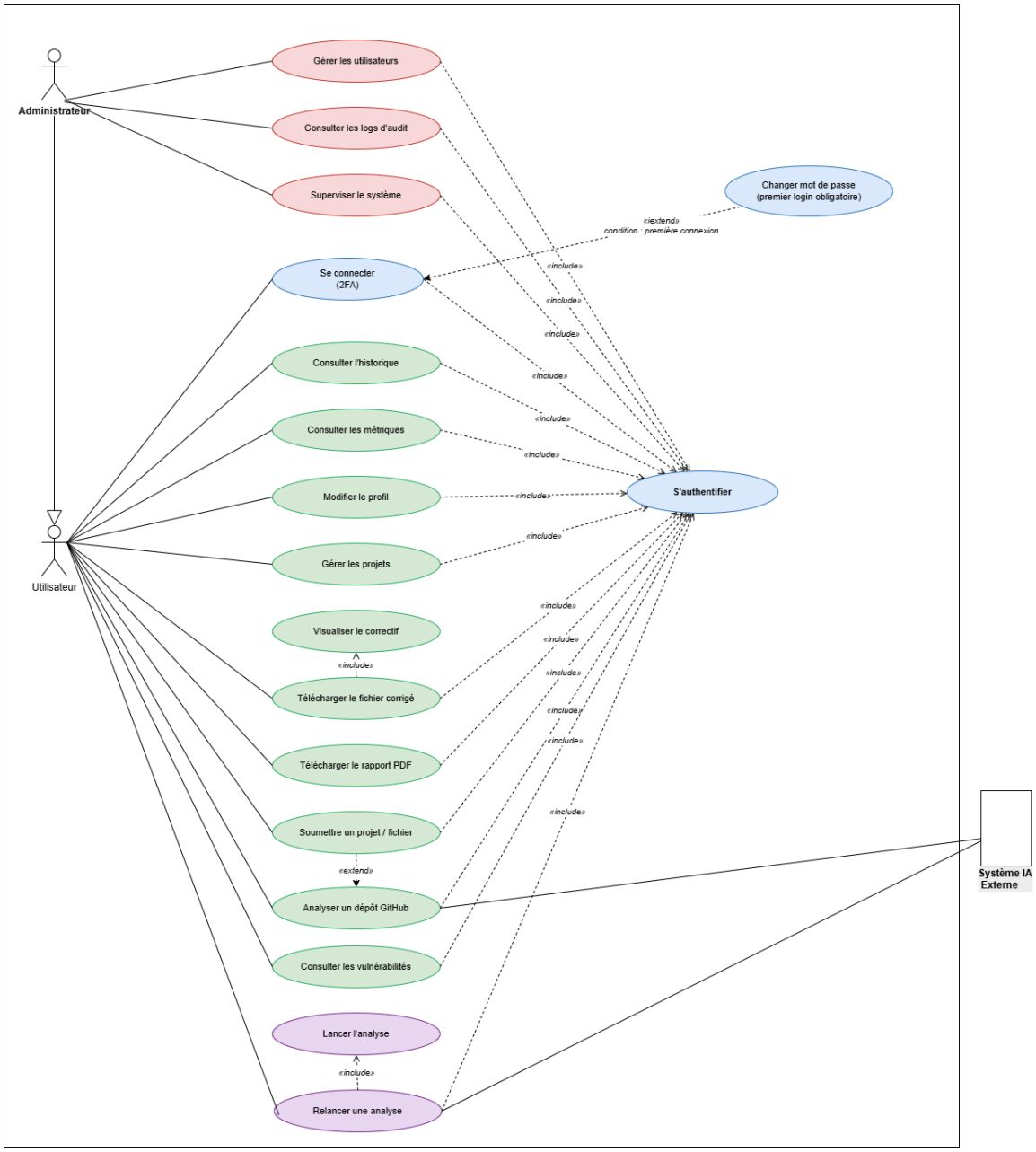


FIGURE 3.3 – Diagramme de cas d'utilisation de PatchMind

3.4.2 Diagramme de classes

Le diagramme de classes représente la structure statique du système et les relations entre les entités principales de la plateforme. Il a été conçu en cohérence avec le modèle de données de la base de données afin de garantir une correspondance directe entre la conception objet et l'implémentation.

La classe centrale *User* est associée aux projets qu'elle possède, aux analyses qu'elle initie, aux commentaires qu'elle rédige et aux journaux d'audit qui tracent ses actions. La classe *Project* regroupe les projets soumis et est liée à plusieurs *Analysis*. Chaque analyse

produit un ensemble de *Vulnerability*, chacune pouvant être associée à un *Patch* généré par le moteur IA.

Les classes *ToolExecution* et *Metric* complètent le modèle en enregistrant respectivement l'exécution des scanners et les métriques calculées pour chaque analyse. La classe *AccessRequest* gère les demandes d'accès à la plateforme, *Comment* permet aux utilisateurs d'annoter les vulnérabilités, *NotificationPreference* centralise les préférences de notification de chaque utilisateur et *AuditLog* assure la traçabilité des actions critiques effectuées sur la plateforme.

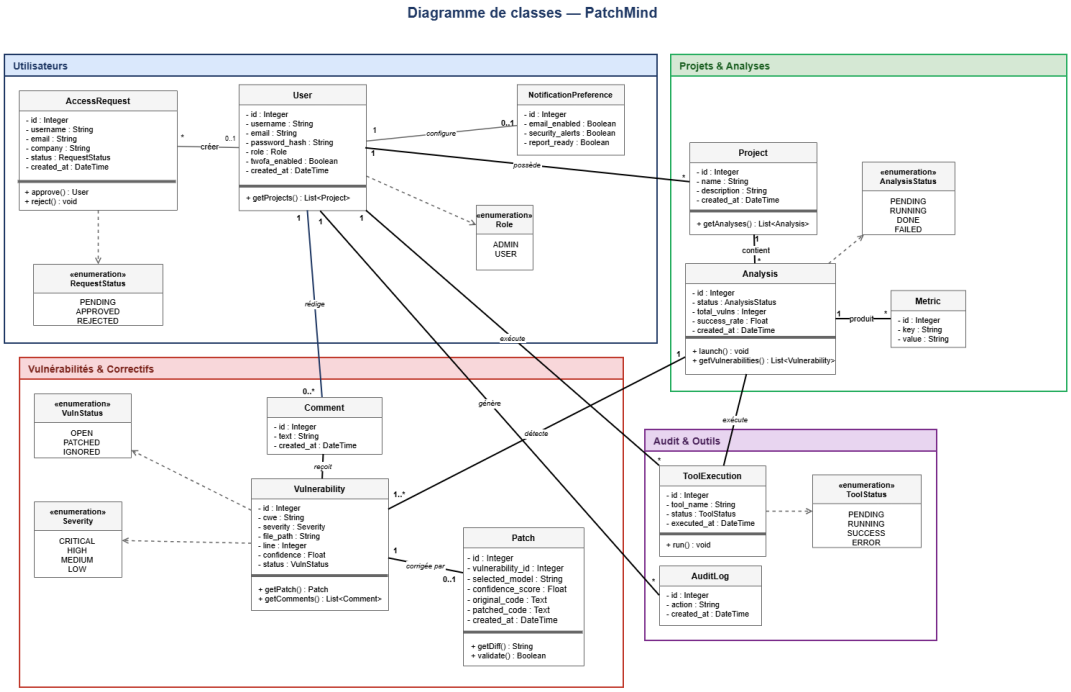


FIGURE 3.4 – Diagramme de classes de PatchMind

3.5 Planification Scrum

Afin d'organiser le développement de la plateforme de manière itérative et progressive, une méthodologie Scrum a été adoptée.

3.5.1 Product Backlog

Le Product Backlog regroupe l'ensemble des fonctionnalités identifiées lors de l'analyse des besoins, organisées par ordre de priorité. Chaque fonctionnalité est exprimée sous

forme de user story décrivant le besoin du point de vue de l'utilisateur ou de l'administrateur. Chaque user story a fait l'objet d'une estimation en points de complexité lors des séances de sprint planning.

Priorité	**User Story**	**Sprint**
Haute	En tant qu'utilisateur, je veux m'authentifier avec la double authentification afin de sécuriser mon accès.	Sprint 1
Haute	En tant qu'utilisateur, je veux changer mon mot de passe lors de ma première connexion afin de sécuriser mon compte.	Sprint 1
Haute	En tant qu'administrateur, je veux gérer les comptes utilisateurs afin de contrôler les accès à la plateforme.	Sprint 1
Haute	En tant qu'utilisateur, je veux soumettre un projet à analyser afin de détecter ses vulnérabilités.	Sprint 2
Haute	En tant qu'utilisateur, je veux consulter les vulnérabilités détectées afin de comprendre les failles présentes dans mon projet.	Sprint 2
Haute	En tant qu'utilisateur, je veux visualiser les correctifs proposés afin de corriger les vulnérabilités détectées.	Sprint 3
Haute	En tant qu'utilisateur, je veux télécharger le fichier corrigé afin d'appliquer directement le correctif proposé.	Sprint 3
Haute	En tant qu'utilisateur, je veux télécharger un rapport PDF afin de conserver les résultats de l'analyse.	Sprint 3
Moyenne	En tant qu'utilisateur, je veux consulter mon historique afin de suivre mes analyses précédentes.	Sprint 3
Moyenne	En tant qu'utilisateur, je veux modifier mon profil afin de mettre à jour mes informations personnelles.	Sprint 4
Moyenne	En tant qu'administrateur, je veux consulter les métriques afin de mesurer les performances globales de la plateforme.	Sprint 4

Priorité	User Story	Sprint
Moyenne	En tant qu'administrateur, je veux consulter le tableau de bord afin de superviser l'activité de la plateforme.	Sprint 4
Moyenne	En tant qu'administrateur, je veux consulter les journaux d'audit afin d'assurer la traçabilité des actions.	Sprint 4
Moyenne	En tant qu'administrateur, je veux superviser le système afin de contrôler l'état général de la plateforme.	Sprint 4
Faible	En tant qu'utilisateur, je veux relancer une analyse afin de vérifier l'évolution de mon projet.	Sprint 4

TABLE 3.1 – Product Backlog du projet PatchMind

3.5.2 Organisation des sprints

Le développement de PatchMind a été organisé en deux releases et quatre sprints, chacun couvrant un ensemble cohérent de fonctionnalités validées à la fin de chaque cycle.

TABLE 3.2 – Organisation des releases et des sprints

Sprint	Release	Fonctionnalités principales	Durée
Sprint 1	Release 1	Authentification, 2FA, RBAC, gestion des utilisateurs	2 semaines
Sprint 2	Release 1	Intégration des scanners, pipeline d'analyse, centralisation des résultats	2 semaines
Sprint 3	Release 2	Génération automatique des correctifs via RAG et consensus multi-LLM, validation des correctifs, rapports PDF, historique des analyses	2 semaines
Sprint 4	Release 2	Dashboard administrateur, audit logs, métriques, tests et validation	2 semaines

3.6 Conclusion

Ce chapitre a présenté l'analyse des besoins ainsi que la conception complète de la plateforme PatchMind. Les besoins fonctionnels et non fonctionnels ont permis de définir précisément les exigences du système. L'architecture modulaire organisée en cinq couches distinctes et le pipeline de traitement en neuf étapes constituent le socle technique de la

solution. Les diagrammes UML modélisent rigoureusement les interactions entre acteurs et la structure interne du système.

Cette phase de conception a permis d'établir une architecture robuste, modulaire et évolutive servant de base à l'implémentation de la plateforme.

Le chapitre suivant est consacré à l'implémentation concrète des fonctionnalités fondamentales de la plateforme, notamment les mécanismes d'authentification, de gestion des accès et le pipeline d'analyse de sécurité.

Chapitre 4

Mise en place du pipeline DevSecOps de PatchMind

4.1 Introduction

La première release de PatchMind a été consacrée à la mise en place des fondations techniques et sécuritaires de la plateforme. Cette étape représente le socle principal du projet, car elle couvre l'ensemble des mécanismes nécessaires au contrôle d'accès, à la gestion des utilisateurs ainsi qu'à l'intégration des outils d'analyse de sécurité.

Cette release est composée de deux sprints principaux. Le premier sprint porte sur l'authentification, la gestion des utilisateurs et la sécurisation globale de la plateforme. Le second sprint concerne l'intégration des scanners de sécurité ainsi que la mise en place du pipeline automatisé d'analyse des vulnérabilités.

4.2 Sprint 1 — Authentification et gestion des utilisateurs

Le premier sprint avait pour objectif de sécuriser l'accès à la plateforme et de mettre en place l'ensemble des fonctionnalités liées à la gestion des utilisateurs. Cette étape était essentielle afin de garantir que seules les personnes autorisées puissent accéder aux fonctionnalités de PatchMind.

4.2.1 Authentification

La première fonctionnalité développée concerne le système d'authentification des utilisateurs, implémenté à l'aide du framework Flask et de la bibliothèque SQLAlchemy pour la gestion des données.

Le backend expose plusieurs routes couvrant :

— l'inscription ;
— la connexion ;
— la déconnexion ;
— la gestion des sessions ;
— la vérification des droits d'accès.

Lors de l'inscription, les informations utilisateur sont validées avant d'être enregistrées dans la base de données SQLite. Les mots de passe ne sont jamais stockés en clair : ils sont automatiquement hachés avant leur enregistrement à l'aide d'un mécanisme sécurisé.

Lors de la connexion, le backend :

1. récupère les informations saisies ;
2. vérifie l'existence du compte ;
3. compare le mot de passe fourni avec le hash stocké ;
4. ouvre une session sécurisée si les informations sont valides.

Cette approche permet de réduire les risques liés au vol ou à la compromission des mots de passe.

La figure suivante présente la page d'accueil publique de la plateforme PatchMind, point d'entrée pour les utilisateurs souhaitant accéder à la plateforme ou demander un accès :

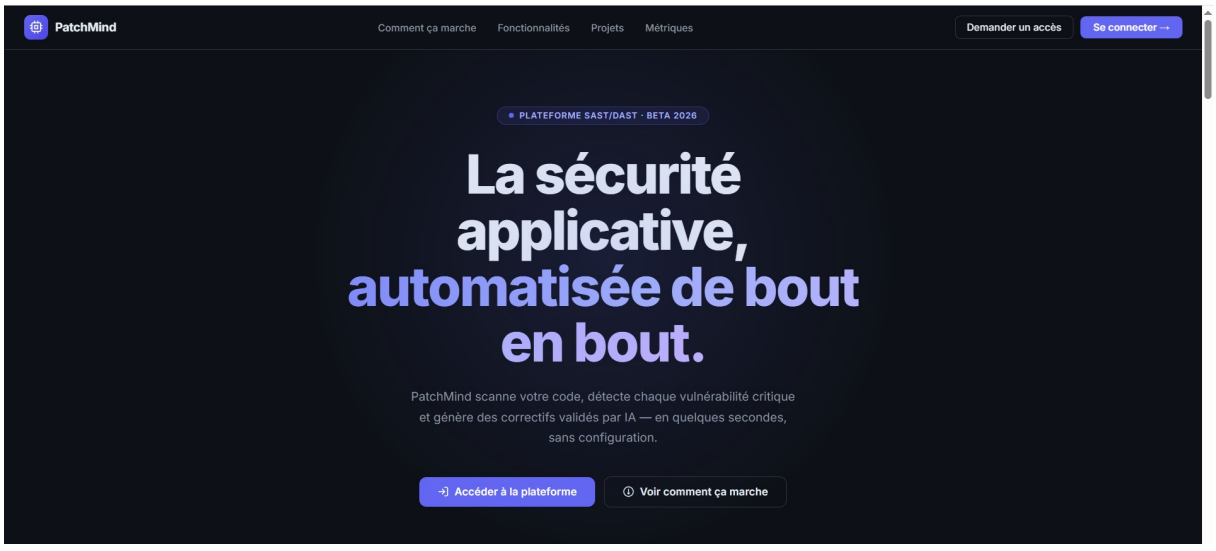


FIGURE 4.1 – Page d'accueil de la plateforme PatchMind

La figure suivante présente l'interface de connexion sécurisée de PatchMind :

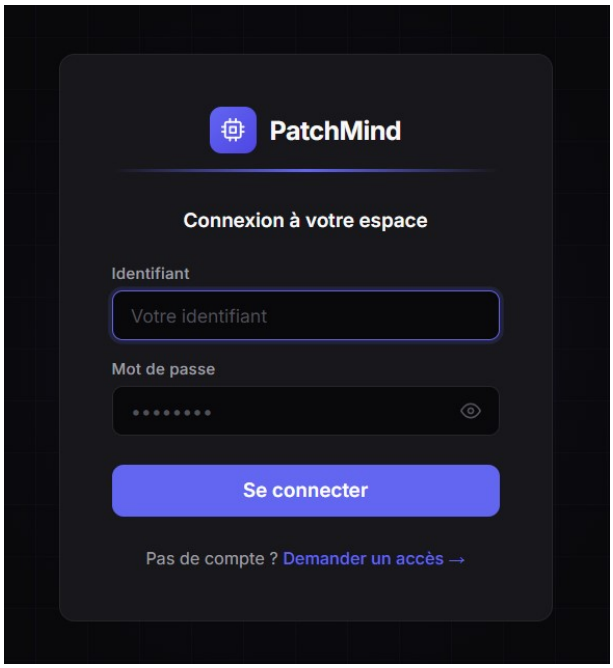


FIGURE 4.2 – Interface de connexion de PatchMind

4.2.2 Gestion des sessions

Après l'authentification, une session utilisateur est créée afin de maintenir l'état de connexion entre les différentes requêtes.

Plusieurs mécanismes de sécurité ont été mis en place :

— limitation de la durée des sessions ;
— invalidation automatique après déconnexion ;
— protection des routes privées ;
— restriction des accès non authentifiés ;
— sécurisation des cookies de session via le flag HTTPOnly.

Le backend Flask vérifie automatiquement l'état de la session avant d'autoriser l'accès aux fonctionnalités sensibles, limitant ainsi les risques liés au détournement de session.

4.2.3 Contrôle d'accès basé sur les rôles (RBAC)

Le contrôle d'accès basé sur les rôles (*Role-Based Access Control* — RBAC) a été implémenté afin de séparer clairement les privilèges des utilisateurs standards et des administrateurs. Chaque utilisateur possède un rôle enregistré dans la base de données, vérifié automatiquement par le backend avant d'autoriser l'accès à certaines fonctionnalités.

Le tableau suivant présente les droits d'accès associés à chaque rôle :

TABLE 4.1 – Droits d'accès par rôle

Fonctionnalité	**Utilisateur**	**Administrateur**
Soumettre des projets et lancer des analyses	✓	✓
Consulter les vulnérabilités et les correctifs	✓	✓
Télécharger les rapports PDF	✓	✓
Modifier son profil utilisateur	✓	✓
Gérer les comptes utilisateurs	–	✓
Consulter les journaux d'audit	–	✓
Accéder au dashboard administrateur	–	✓
Superviser l'état général du système	–	✓

Cette séparation des rôles permet de limiter l'accès aux fonctionnalités critiques aux seuls profils autorisés.

4.2.4 Double authentification (2FA)

Afin de renforcer davantage la sécurité des comptes utilisateurs, un système de double authentification (*2FA*) a été intégré, ajoutant une seconde étape de validation après la saisie du mot de passe. La génération des codes temporaires repose sur le protocole TOTP (*Time-based One-Time Password*), implémenté à l'aide de la bibliothèque PyOTP et compatible avec les applications d'authentification standard telles que Google Authenticator.

Le fonctionnement se déroule comme suit :

1. l'utilisateur saisit son email et son mot de passe ;
2. le backend vérifie les identifiants ;
3. si les informations sont correctes, une seconde étape de validation est déclenchée ;
4. l'utilisateur fournit un code temporaire généré automatiquement ;
5. le backend vérifie la validité du code ;
6. la session est ouverte uniquement si le code est valide.

Cette approche réduit considérablement les risques liés au vol de mots de passe et aux accès non autorisés.

La figure suivante présente l'interface de configuration de la double authentification, guidant l'utilisateur à travers les trois étapes d'activation :

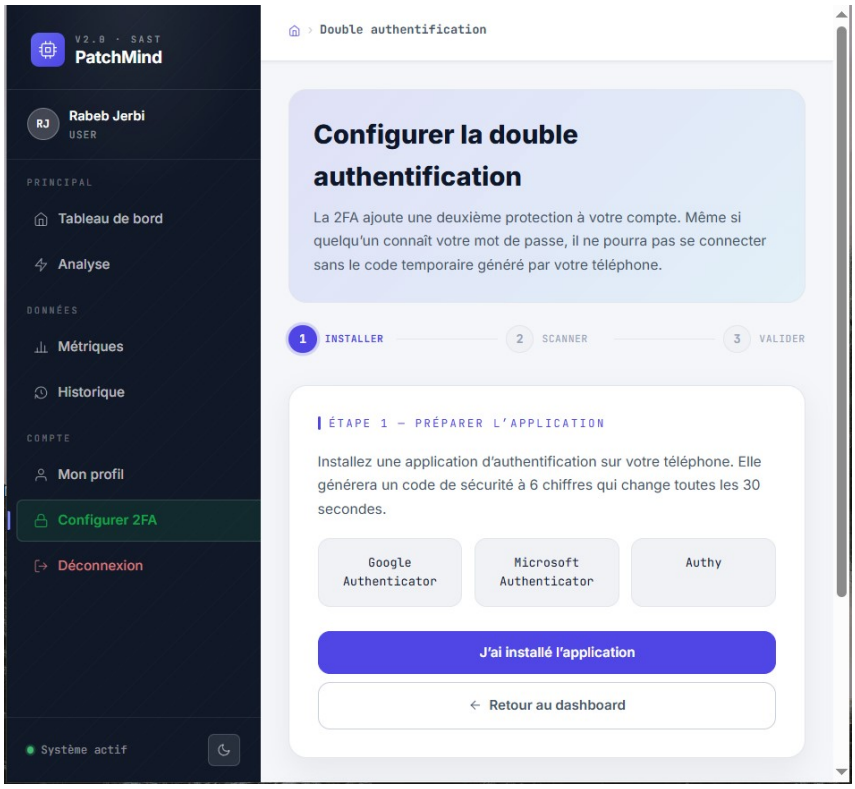


FIGURE 4.3 – Configuration de la double authentification (TOTP)

4.2.5 Validation des demandes d'accès

Les demandes d'accès à la plateforme sont soumises à une validation administrative avant la création définitive du compte utilisateur.

Lorsqu'un administrateur approuve une demande d'inscription, les informations de connexion de l'utilisateur sont automatiquement générées puis envoyées par courrier électronique.

Lors de la première connexion, l'utilisateur est obligé de modifier son mot de passe afin de garantir la sécurité du compte et d'éviter l'utilisation prolongée des identifiants temporaires générés automatiquement.

Ce mécanisme permet de renforcer le contrôle d'accès à la plateforme tout en améliorant la sécurité des comptes utilisateurs.

4.2.6 Sécurisation de la plateforme

En complément des mécanismes d'authentification, plusieurs mesures de sécurité supplémentaires ont été mises en place afin de renforcer la protection globale de PatchMind.

Les principales protections implémentées sont :

— validation et assainissement des entrées utilisateur afin de prévenir les injections et les attaques par manipulation de paramètres ;
— protection des routes sensibles garantissant qu'aucune fonctionnalité critique n'est accessible sans authentification préalable et vérification du rôle ;
— limitation des privilèges administrateur aux seules fonctionnalités de supervision ;
— sécurisation des sessions via le flag HTTPOnly et la limitation de leur durée de vie ;
— journalisation des actions critiques assurant la traçabilité complète des opérations réalisées sur la plateforme ;
— chiffrement des données sensibles stockées en base de données.

4.2.7 Interfaces utilisateur

Plusieurs interfaces web ont été développées durant ce sprint afin de permettre l'interaction avec les fonctionnalités d'authentification et de gestion des utilisateurs.

Les principales interfaces réalisées couvrent :

— la page de connexion ;
— le tableau de bord utilisateur ;
— la gestion du profil utilisateur ;
— la configuration du 2FA ;
— l'espace administrateur.

Une attention particulière a été accordée à l'ergonomie, à la simplicité de navigation et à la cohérence visuelle de l'ensemble des pages.

La figure suivante présente le tableau de bord utilisateur centralisant les analyses effectuées, les vulnérabilités détectées et les métriques globales :

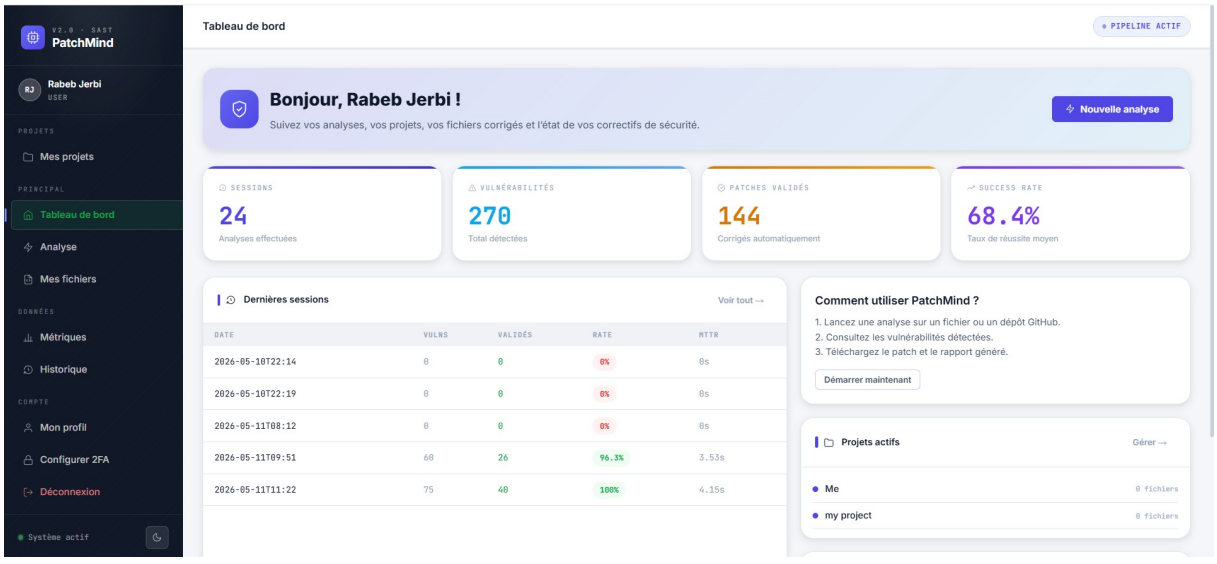


FIGURE 4.4 – Tableau de bord utilisateur de PatchMind

La figure suivante présente l'interface de gestion du profil utilisateur, permettant la modification des informations personnelles et la sécurisation du compte :

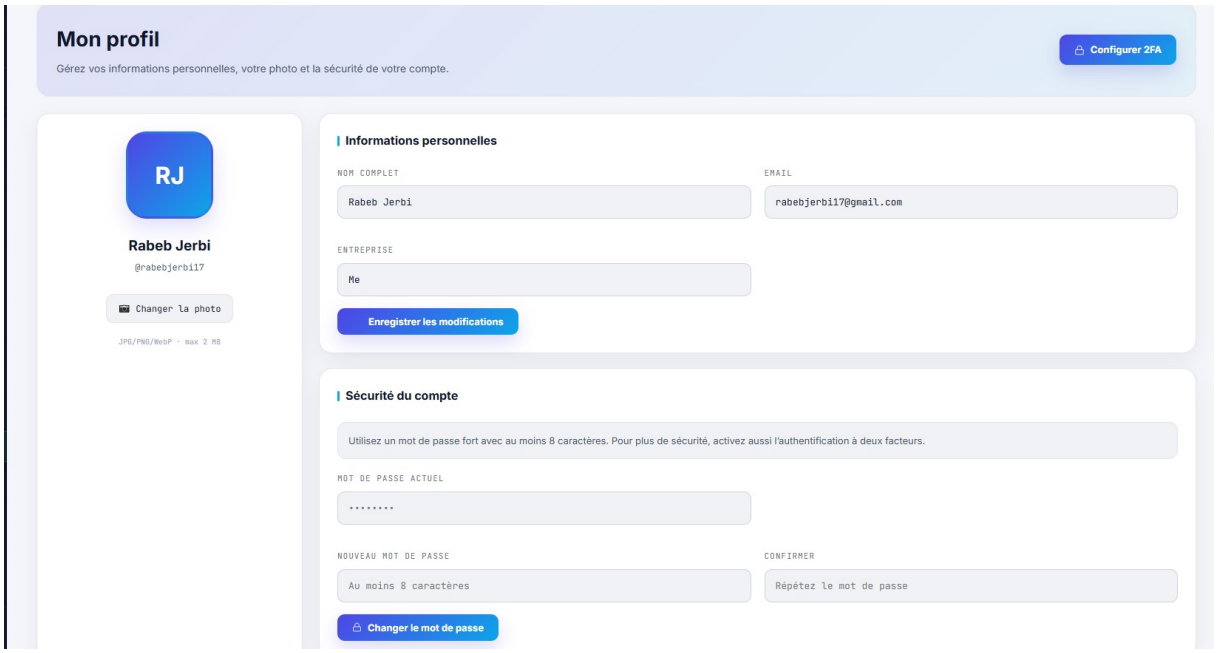


FIGURE 4.5 – Interface de gestion du profil utilisateur

La figure suivante présente l'interface de gestion des projets, permettant à l'utilisateur d'organiser et de suivre ses bases de code analysées :

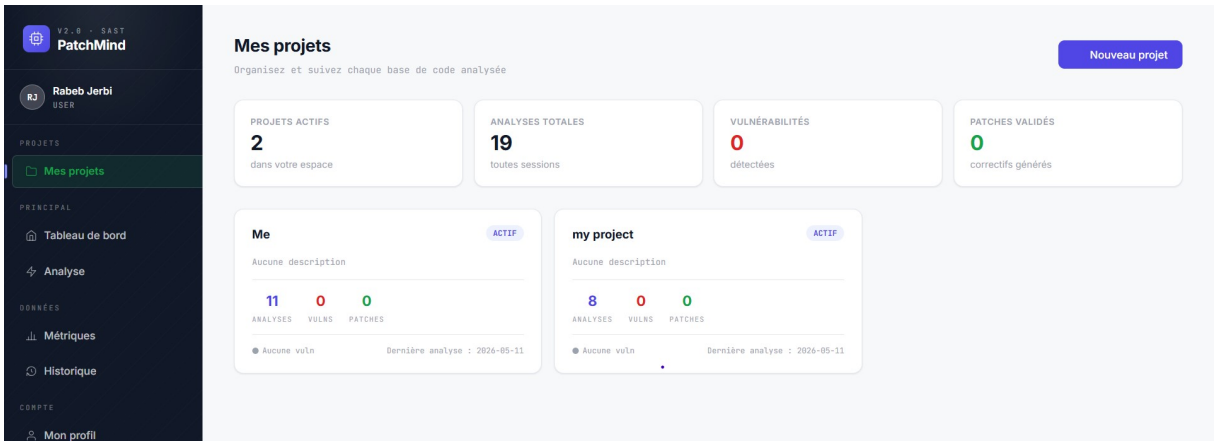


FIGURE 4.6 – Interface de gestion des projets utilisateur

4.3 Sprint 2 — Intégration des outils de sécurité

Le second sprint avait pour objectif d'intégrer les différents outils d'analyse de sécurité et de mettre en place le pipeline automatisé de traitement des vulnérabilités.

Cette étape constitue l'un des composants centraux de PatchMind, car elle permet

d'automatiser la détection des vulnérabilités applicatives à partir des projets soumis par les utilisateurs. Quatre scanners principaux ont été intégrés, couvrant l'analyse statique, la détection de secrets, l'audit des dépendances et les tests dynamiques.

4.3.1 Architecture du pipeline de scan

Le pipeline de scan a été conçu afin d'orchestrer automatiquement plusieurs outils d'analyse de sécurité à partir d'un projet soumis par l'utilisateur.

Lorsqu'un projet est envoyé à la plateforme, le backend :

1. identifie les fichiers et technologies utilisés ;
2. sélectionne les scanners adaptés au type de projet ;
3. exécute les outils de manière coordonnée ;
4. récupère et parse leurs résultats bruts ;
5. centralise les vulnérabilités détectées ;
6. stocke les résultats dans la base de données.

La persistance des données est assurée par SQLite via l'ORM SQLAlchemy.

Cette architecture permet d'unifier les analyses de sécurité au sein d'une interface unique et cohérente, indépendamment du format de sortie de chaque outil.

La figure suivante présente l'interface de soumission d'un fichier à analyser, avec la sélection des scanners à utiliser ainsi que les outils optionnels disponibles :

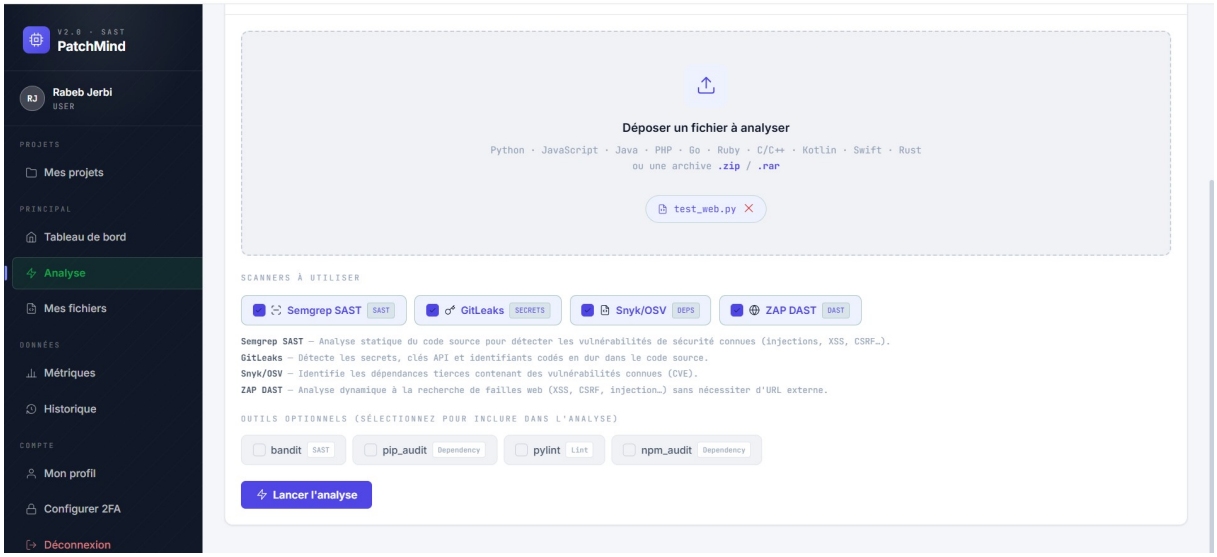


FIGURE 4.7 – Interface de soumission d'un fichier et sélection des scanners

4.3.2 Intégration de Semgrep

Le premier scanner intégré dans PatchMind est Semgrep, utilisé pour réaliser l'analyse statique du code source (*SAST*).

Un module Python dédié automatise l'exécution de Semgrep sur les fichiers soumis puis parse les résultats au format JSON afin d'extraire :

— le nom de la vulnérabilité ;
— le niveau de sévérité ;
— le fichier concerné ;
— la ligne affectée ;
— les références CWE associées.

Semgrep permet notamment de détecter :

— les injections SQL (CWE-89) ;
— les vulnérabilités XSS (CWE-79) ;
— les erreurs de validation des entrées (CWE-20) ;
— certaines failles de l'OWASP Top 10.

Les vulnérabilités détectées sont ensuite stockées dans la base de données et affichées dans l'interface utilisateur.

4.3.3 Intégration de GitLeaks

GitLeaks a été intégré afin de détecter les secrets exposés dans les fichiers source ou les historiques de dépôts Git.

Son objectif principal est d'identifier :

— les clés API ;
— les mots de passe ;
— les jetons d'accès ;
— les informations sensibles exposées accidentellement.

La fuite de telles informations pouvant compromettre l'ensemble d'un système, cette analyse constitue une étape critique du pipeline de sécurité.

4.3.4 Intégration de Snyk

Snyk a été utilisé afin d'analyser les dépendances et les bibliothèques tierces utilisées dans les projets soumis.

Son intégration permet :

— d'analyser les fichiers de dépendances ;
— d'identifier les composants vulnérables ;
— de récupérer les références CVE associées ;
— de classer les vulnérabilités selon leur sévérité.

Cette intégration réduit les risques liés aux bibliothèques open source vulnérables et aux attaques de type Supply Chain.

4.3.5 Intégration d'OWASP ZAP

OWASP ZAP a été intégré afin de réaliser des analyses dynamiques (*DAST*) des applications web.

Contrairement aux outils SAST, ZAP analyse l'application en cours d'exécution et permet d'identifier :

— les problèmes de gestion des sessions ;
— les erreurs de configuration ;
— les failles exploitables dynamiquement ;
— certaines vulnérabilités invisibles à l'analyse statique.

Le backend orchestre automatiquement :

— le lancement des scans ;
— la récupération des alertes ;
— le parsing des résultats ;
— leur intégration dans le système centralisé.

4.3.6 Outils complémentaires optionnels

En complément des scanners principaux, PatchMind permet également l'intégration optionnelle de plusieurs outils spécialisés afin d'enrichir les analyses de sécurité selon les technologies détectées dans les projets soumis.

Les outils complémentaires intégrés sont :

— **Bandit** : analyse statique des vulnérabilités spécifiques aux applications Python ;
— **pip-audit** : détection des dépendances Python vulnérables ;
— **pylint** : analyse de qualité et de bonnes pratiques du code Python ;
— **npm-audit** : audit de sécurité des dépendances Node.js.

Ces outils sont visibles dans l'interface de soumission présentée précédemment et peuvent être sélectionnés selon les technologies utilisées dans le projet analysé.

4.3.7 Centralisation des résultats

L'un des principaux défis de ce sprint concernait la normalisation des résultats produits par les différents scanners. Chaque outil possédant son propre format de sortie, un mécanisme de centralisation a été développé afin d'unifier les données.

Chaque vulnérabilité est transformée dans une structure commune contenant :

— le type de vulnérabilité ;
— le niveau de sévérité ;
— le fichier concerné ;
— la ligne affectée ;
— le scanner détecteur ;
— les références CWE et CVE ;
— le statut de remédiation.

Cette normalisation facilite :

— l'affichage unifié des résultats ;
— la génération des rapports ;
— l'exploitation des données par le moteur IA ;
— le stockage structuré des analyses.

La figure suivante présente l'affichage centralisé des vulnérabilités détectées après exécution du pipeline :

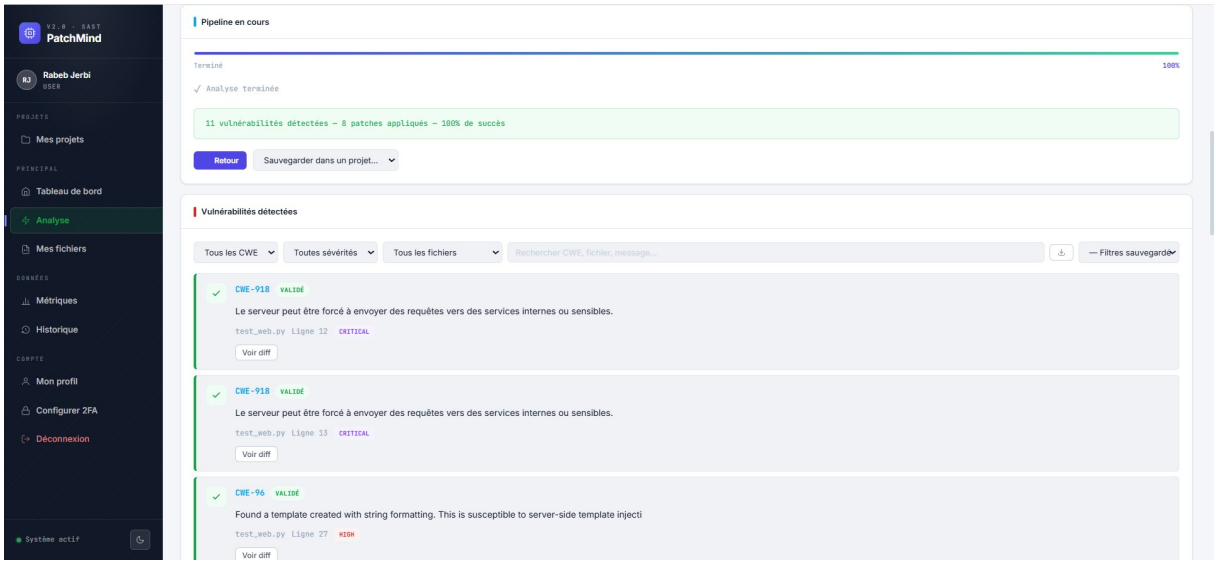


FIGURE 4.8 – Affichage centralisé des vulnérabilités détectées

4.3.8 Exécution du pipeline d'analyse

Après la soumission d'un projet, PatchMind lance automatiquement les différents scanners sélectionnés et affiche en temps réel l'état d'avancement du pipeline d'analyse.

L'utilisateur peut suivre :

— l'exécution des scanners ;

— le parsing des résultats ;

— la centralisation des vulnérabilités détectées ;

— la progression globale du pipeline.

Cette fonctionnalité améliore l'expérience utilisateur et permet de visualiser clairement les différentes étapes de traitement des vulnérabilités.

La figure suivante présente le pipeline d'analyse en cours d'exécution, affichant la progression en temps réel ainsi que le résumé des résultats obtenus :

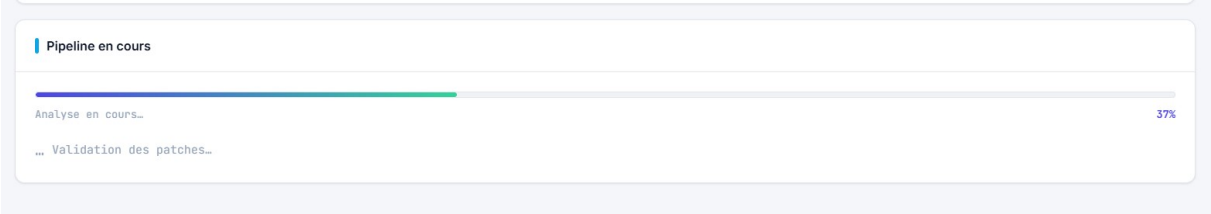


FIGURE 4.9 – Pipeline d'analyse en cours d'exécution

4.3.9 Résultats des analyses

Une fois les analyses terminées, PatchMind affiche les vulnérabilités détectées par les différents scanners. Chaque vulnérabilité contient :

— sa référence CWE ;

— son niveau de sévérité ;

— le fichier concerné ;

— la ligne affectée ;

— le scanner ayant effectué la détection.

Ces résultats constituent l'entrée du moteur de remédiation intelligente présenté au chapitre suivant.

La figure suivante présente la liste des vulnérabilités détectées avec leur statut de correction et la possibilité de visualiser le diff associé à chaque correctif :

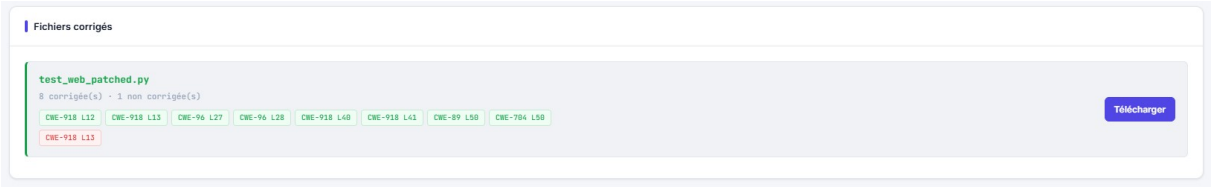


FIGURE 4.10 – Affichage des vulnérabilités détectées et de leur statut de correction

4.3.10 Automatisation du pipeline

Dans le cadre de l'approche DevSecOps, un pipeline GitHub Actions a également été configuré afin d'automatiser certaines tâches de sécurité et de validation.

Cette automatisation permet notamment :

— l'exécution automatique des scanners ;

— le lancement des analyses de sécurité ;

— la vérification des dépendances ;

— l'amélioration de l'intégration continue du projet.

Cette approche facilite l'intégration des mécanismes de sécurité directement dans le cycle de développement logiciel, conformément aux principes DevSecOps.

4.3.11 Difficultés rencontrées

Durant ce sprint, plusieurs difficultés techniques ont été rencontrées.

La principale concernait la gestion des formats de sortie hétérogènes des scanners de sécurité. Chaque outil possédant sa propre structure de données, la conception du mécanisme de normalisation a nécessité une analyse approfondie avant de définir un modèle commun suffisamment flexible.

Des difficultés liées aux performances ont également été observées lors de l'exécution coordonnée de plusieurs scanners, nécessitant des optimisations afin de limiter le temps global d'analyse.

L'intégration des différents outils requérait par ailleurs une gestion rigoureuse des dépendances et des environnements d'exécution afin d'assurer la compatibilité entre les scanners et le backend Flask.

L'intégration des modèles LLM a également nécessité plusieurs ajustements concernant le formatage des prompts, la validation des réponses générées et la réduction des faux correctifs produits automatiquement.

4.4 Conclusion

Cette première release a permis de mettre en place les fondations techniques et sécuritaires de la plateforme PatchMind.

Le premier sprint a établi un système d'authentification robuste intégrant le hachage sécurisé des mots de passe, la gestion des sessions, le contrôle d'accès basé sur les rôles (RBAC) et la double authentification via le protocole TOTP.

Le second sprint a permis d'intégrer quatre scanners de sécurité complémentaires — Semgrep, GitLeaks, Snyk et OWASP ZAP — couvrant respectivement l'analyse statique, la détection de secrets, l'analyse des dépendances et les tests dynamiques, enrichis par plusieurs outils optionnels adaptés aux technologies détectées. Un pipeline automatisé a également été développé afin d'orchestrer les analyses et de centraliser les vulnérabilités détectées dans un format unifié exploitable par le moteur IA. L'ensemble du pipeline a par ailleurs été intégré dans un workflow GitHub Actions, renforçant ainsi la dimension DevSecOps de la plateforme.

Le chapitre suivant est consacré à l'implémentation du moteur intelligent de remédiation, détaillant les mécanismes de génération automatique des correctifs, le consensus multi-LLM, la validation des correctifs ainsi que les fonctionnalités de reporting et de supervision administrative.

Cette release constitue la base opérationnelle de PatchMind et prépare l'intégration des mécanismes avancés d'intelligence artificielle, de génération automatique de correctifs et de supervision présentés dans la release suivante.

Chapitre 5

Automatisation de la remédiation et supervision de la plateforme

5.1 Introduction

La deuxième release de PatchMind a été consacrée aux fonctionnalités avancées de la plateforme. Après la mise en place des mécanismes d'authentification et du pipeline d'analyse de sécurité durant la première release, cette phase a permis d'enrichir la plateforme avec des fonctionnalités de génération intelligente de correctifs, de reporting structuré, de supervision administrative et de gestion des analyses.

Cette release couvre également les différents tests réalisés afin de vérifier la stabilité, la sécurité et les performances globales de la plateforme, ainsi que les résultats obtenus à l'issue du développement complet de PatchMind.

5.2 Sprint 3 — Remédiation intelligente et reporting

Le troisième sprint avait pour objectif de développer les fonctionnalités liées à la génération intelligente des correctifs, à la production des rapports d'analyse, au stockage des résultats ainsi qu'à la gestion de l'historique des analyses réalisées par les utilisateurs.

5.2.1 Génération intelligente des correctifs

Après la détection des vulnérabilités par le pipeline de scan, PatchMind exploite les résultats collectés pour générer automatiquement des propositions de correction. Le moteur de génération construit un contexte riche à partir de la description de la vulnérabilité,

du fichier concerné, de l'extrait de code vulnérable, du niveau de sévérité, des références CWE ou CVE et des exemples similaires retrouvés dans la base de connaissances.

L'architecture RAG intégrée améliore la qualité des réponses générées. Son fonctionnement suit plusieurs étapes : les exemples de vulnérabilités et de correctifs sont transformés en représentations vectorielles (*embeddings*) par la bibliothèque Sentence Transformers, permettant de représenter les descriptions de vulnérabilités sous forme de vecteurs sémantiques comparables, puis indexés avec FAISS. Lorsqu'une vulnérabilité est détectée, PatchMind recherche les exemples les plus proches dans l'index vectoriel, les ajoute au prompt envoyé au modèle IA, puis le modèle génère un correctif en tenant compte de ce contexte enrichi.

Pour améliorer la fiabilité de la génération, PatchMind adopte une approche de consensus multi-LLM. Plusieurs propositions sont produites indépendamment par plusieurs modèles de langage exécutés via Groq API, Gemini et Ollama, puis comparées selon des critères de cohérence, de similarité, de conformité syntaxique et de capacité à supprimer la vulnérabilité. Le correctif présentant le meilleur score de consensus est sélectionné comme proposition principale et présenté à l'utilisateur avec son score de confiance.

La figure suivante illustre le fonctionnement du moteur IA de PatchMind, depuis la détection de la vulnérabilité jusqu'à la sélection du correctif final :

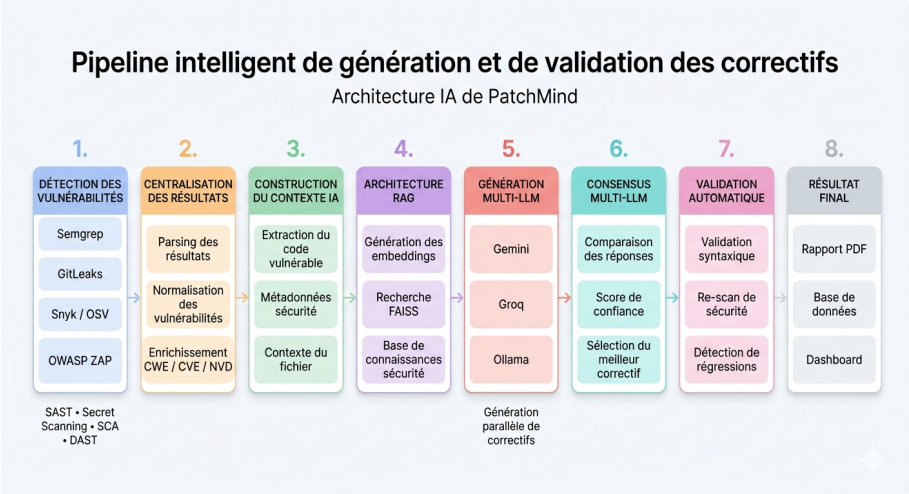


FIGURE 5.1 – Pipeline du moteur IA de génération et de sélection des correctifs

La figure suivante présente le diff avant/après correction généré automatiquement par PatchMind pour une vulnérabilité de type injection SQL (CWE-89) :

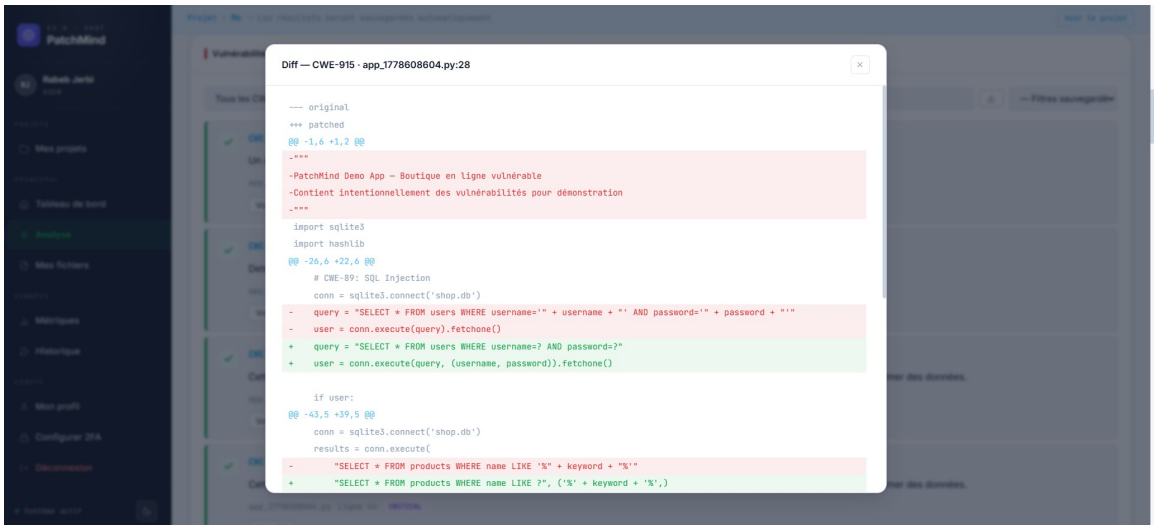


FIGURE 5.2 – Visualisation du diff avant/après correction généré par PatchMind

La figure suivante présente l'interface des fichiers corrigés, affichant les références CWE corrigées par ligne ainsi que le bouton de téléchargement :

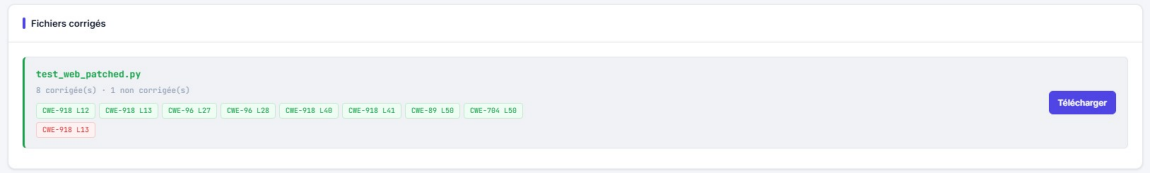


FIGURE 5.3 – Interface des fichiers corrigés avec les références CWE associées

5.2.2 Validation automatique des correctifs

Après la génération des correctifs, une étape de validation automatique a été mise en place afin de vérifier que le correctif généré reste syntaxiquement valide et ne crée pas de nouvelles régressions.

Quatre mécanismes de validation sont appliqués :

— vérification syntaxique du code selon le langage utilisé ;

— re-scan de sécurité pour confirmer la suppression de la vulnérabilité ;

— détection de régressions par comparaison entre le code original et le code corrigé ;

— calcul d'un score de confiance global évaluant la fiabilité du correctif.

Cette validation multicouche réduit le risque de proposer à l'utilisateur un correctif invalide ou contre-productif.

5.2.3 Outils complémentaires

PatchMind supporte également plusieurs outils optionnels permettant d'enrichir l'analyse de sécurité selon le type de projet analysé :

— **Bandit** : analyse de sécurité spécifique aux projets Python ;
— **pip-audit** : détection des vulnérabilités dans les dépendances Python ;
— **pylint** : analyse de qualité et de bonnes pratiques du code Python ;
— **npm audit** : détection des vulnérabilités dans les dépendances JavaScript et Node.js.

Ces outils permettent d'adapter les analyses au contexte technologique de chaque projet soumis, renforçant ainsi la flexibilité et la couverture de la plateforme.

5.2.4 Intégration CI/CD

Dans le cadre de l'approche DevSecOps, un pipeline GitHub Actions a été configuré sous le nom *PatchMind Security Scan* afin d'automatiser l'exécution des analyses de sécurité lors des phases de développement. Le workflow est déclenché automatiquement à chaque push sur les branches `main`, `master` et `develop`, ainsi qu'à chaque pull request.

Le pipeline CI/CD permet notamment :

— l'installation automatique des dépendances et la configuration de l'environnement ;
— l'exécution du scan PatchMind sur l'ensemble du code source ;
— la publication des résultats sous forme de commentaire automatique sur les pull requests ;
— l'analyse de l'image Docker avec Trivy pour détecter les vulnérabilités critiques ;
— l'échec automatique du pipeline en cas de vulnérabilités critiques non corrigées.

Au total, **12 exécutions** du workflow ont été réalisées durant le développement du projet, avec un temps d'exécution moyen de **6 minutes 42 secondes**.

La figure suivante présente l'historique des exécutions du pipeline CI/CD GitHub Actions de PatchMind :

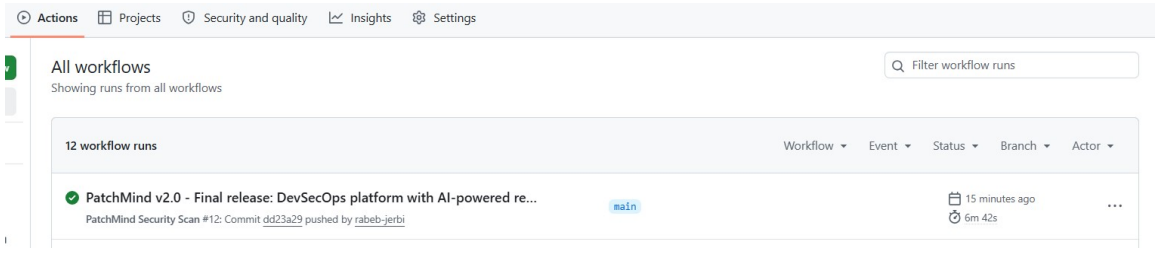


FIGURE 5.4 – Historique des exécutions du pipeline CI/CD GitHub Actions de PatchMind

5.2.5 Génération des rapports PDF

L'une des fonctionnalités développées durant ce sprint concerne la génération automatique des rapports PDF à l'issue de chaque analyse de sécurité.

Le système récupère automatiquement l'ensemble des informations produites durant l'analyse :

— les vulnérabilités détectées ;
— les niveaux de sévurité ;
— les références CWE et CVE ;
— les correctifs générés ;
— les scores de confiance ;
— les résultats de validation.

Ces informations sont organisées dans un document structuré comprenant une page de résumé, les statistiques globales de l'analyse, la liste détaillée des vulnérabilités et les correctifs proposés. Le rapport ainsi généré facilite le partage des analyses entre les membres d'une équipe de développement ou de sécurité, ainsi que leur exploitation dans une démarche d'audit ou de conformité.

La figure suivante présente la page de garde du rapport PDF généré automatiquement par PatchMind :

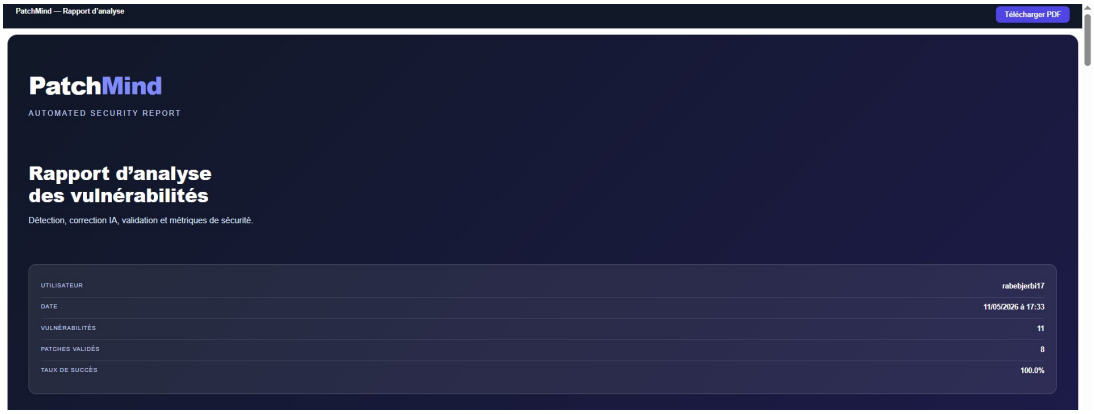


FIGURE 5.5 – Page de garde du rapport d'analyse généré par PatchMind

La figure suivante présente le contenu détaillé du rapport, incluant l'analyse des vulnérabilités avec le diff avant/après correction et les métriques globales de session :

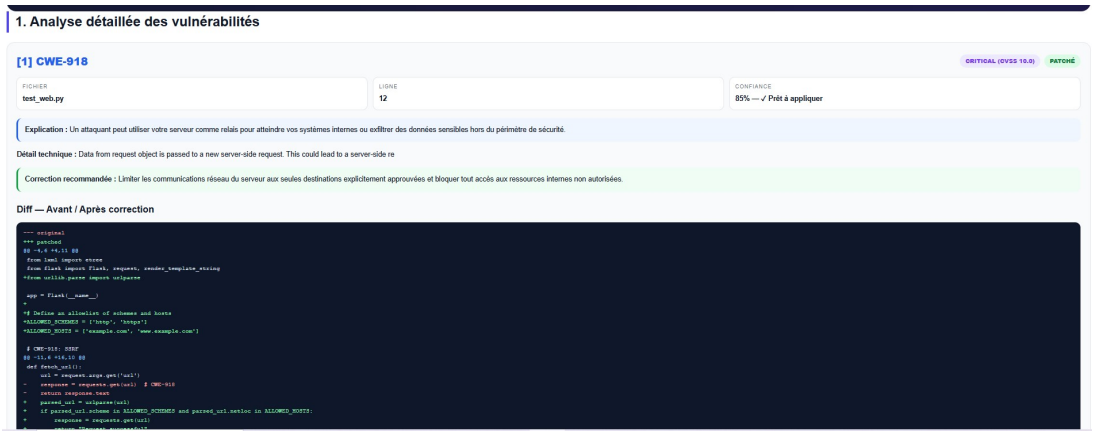


FIGURE 5.6 – Contenu détaillé du rapport d'analyse — vulnérabilités et métriques

5.2.6 Gestion de l'historique

Afin d'améliorer la traçabilité des analyses réalisées, un système d'historique a été implémenté. Chaque analyse exécutée est automatiquement enregistrée dans la base de données avec les informations du projet, la date d'exécution, les vulnérabilités détectées, les correctifs générés, les rapports associés et le statut global de l'analyse.

L'historique offre aux utilisateurs la possibilité :

— de retrouver les analyses précédentes ;
— de suivre l'évolution de la posture de sécurité d'un projet ;
— de comparer plusieurs résultats dans le temps ;
— d'assurer une meilleure traçabilité des corrections appliquées.

La figure suivante présente l'interface de gestion de l'historique des analyses avec le détail des fichiers analysés et les actions disponibles :

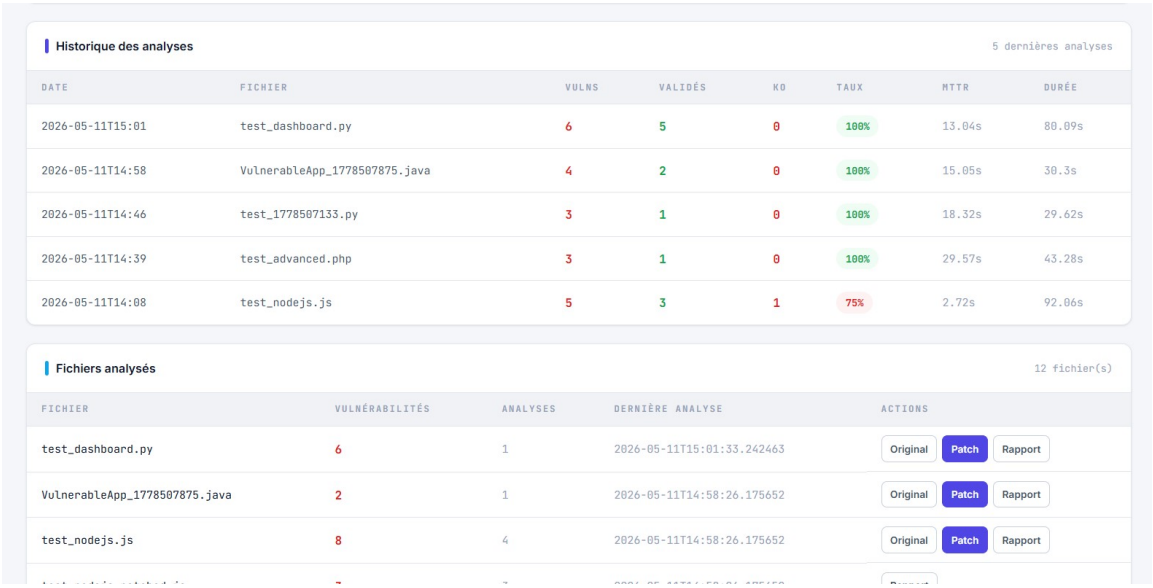


FIGURE 5.7 – Interface de l'historique des analyses

5.2.7 Gestion collaborative des projets

PatchMind permet également la gestion collaborative des projets de sécurité analysés sur la plateforme.

Un utilisateur peut créer un projet dédié afin de centraliser les analyses, les vulnérabilités détectées ainsi que les correctifs générés associés à un même dépôt ou une même application. Le propriétaire du projet peut inviter d'autres utilisateurs à collaborer via un système d'invitation par courrier électronique.

Deux scénarios sont pris en charge :

— si l'utilisateur invité possède déjà un compte PatchMind, une demande d'ajout au projet est transmise à l'administrateur puis l'utilisateur est automatiquement ajouté après validation ;

— si l'utilisateur invité ne possède pas encore de compte, un compte temporaire est automatiquement créé et les informations de connexion sont envoyées par courrier électronique après validation administrative.

Cette approche facilite le travail collaboratif entre développeurs, analystes sécurité et administrateurs au sein d'un même projet.

La figure suivante présente l'interface de gestion collaborative d'un projet, affichant les membres actifs, les invitations en attente ainsi que les informations générales du projet :

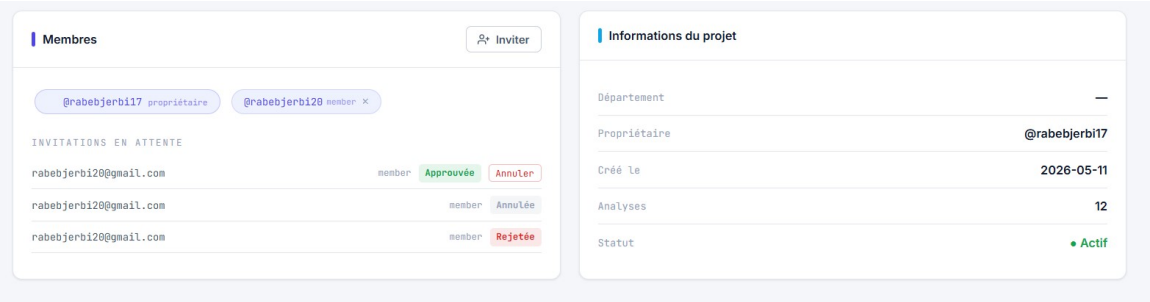


FIGURE 5.8 – Interface de gestion collaborative des projets PatchMind

5.2.8 Stockage des analyses

Le stockage des analyses constitue un composant essentiel de PatchMind. Les résultats produits par les scanners ainsi que les correctifs générés sont automatiquement sauvegardés dans la base de données SQLite via SQLAlchemy. SQLite a été choisi pour sa simplicité d'intégration, sa légèreté et son adaptation aux besoins du prototype PatchMind.

Les données sont réparties dans plusieurs tables relationnelles correspondant aux entités définies lors de la conception :

— projets ;
— analyses ;
— vulnérabilités ;
— correctifs ;
— métriques ;
— journaux d'audit.

Cette organisation garantit une consultation rapide des résultats, une meilleure traçabilité, la génération de statistiques et l'exploitation des données par le dashboard administrateur.

5.2.9 Consultation des résultats

Une interface dédiée a été développée afin de permettre aux utilisateurs de consulter les résultats des analyses réalisées.

L'interface offre :

— l'affichage structuré des vulnérabilités détectées ;
— la visualisation des niveaux de sévérité ;
— la consultation des correctifs générés avec leur score de confiance ;
— le téléchargement des rapports PDF ;

— l'accès à l'historique complet des analyses.

Les résultats sont présentés de manière claire et hiérarchisée afin d'améliorer la lisibilité et de faciliter l'exploitation des informations de sécurité par les développeurs comme par les responsables sécurité.

La figure suivante présente l'interface de suivi d'un projet avec l'évolution des vulnérabilités dans le temps :

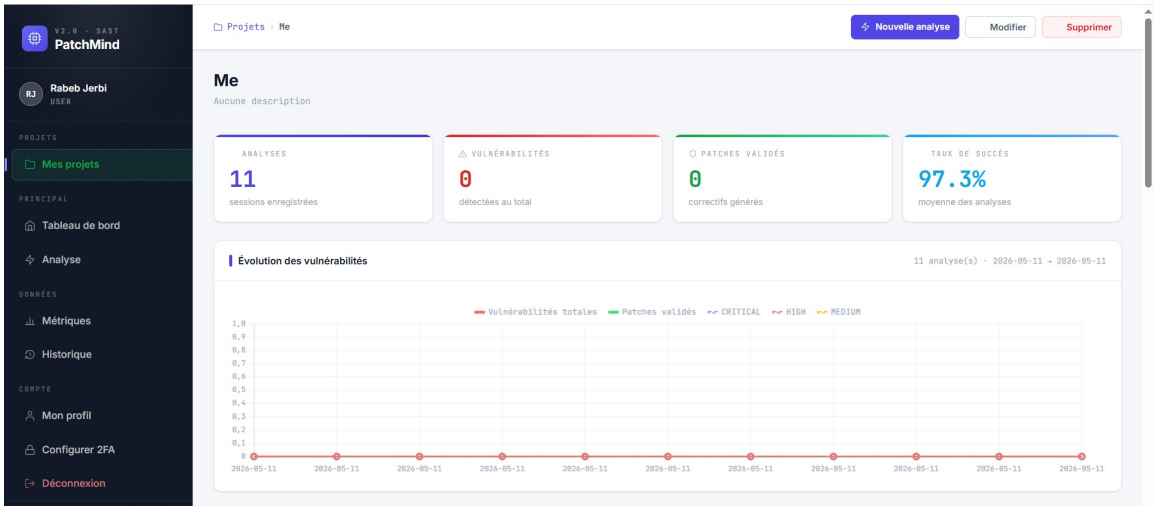


FIGURE 5.9 – Interface de suivi d'un projet et évolution des vulnérabilités

5.3 Sprint 4 — Dashboard administrateur et tests

Le quatrième sprint a été consacré à la supervision globale de la plateforme ainsi qu'à la validation complète des fonctionnalités développées tout au long du projet.

5.3.1 Dashboard administrateur

Un dashboard administrateur a été développé afin de fournir une vue centralisée sur l'activité globale de la plateforme.

Le dashboard permet :

— la supervision des comptes utilisateurs ;

— le suivi des analyses exécutées ;

— la consultation des vulnérabilités détectées ;

— l'accès aux métriques globales ;

— la consultation des journaux d'audit.

L'objectif principal était de centraliser les informations importantes dans une interface unique accessible aux administrateurs.

La figure suivante présente le dashboard administrateur de PatchMind centralisant l'ensemble des métriques et de l'activité de la plateforme :

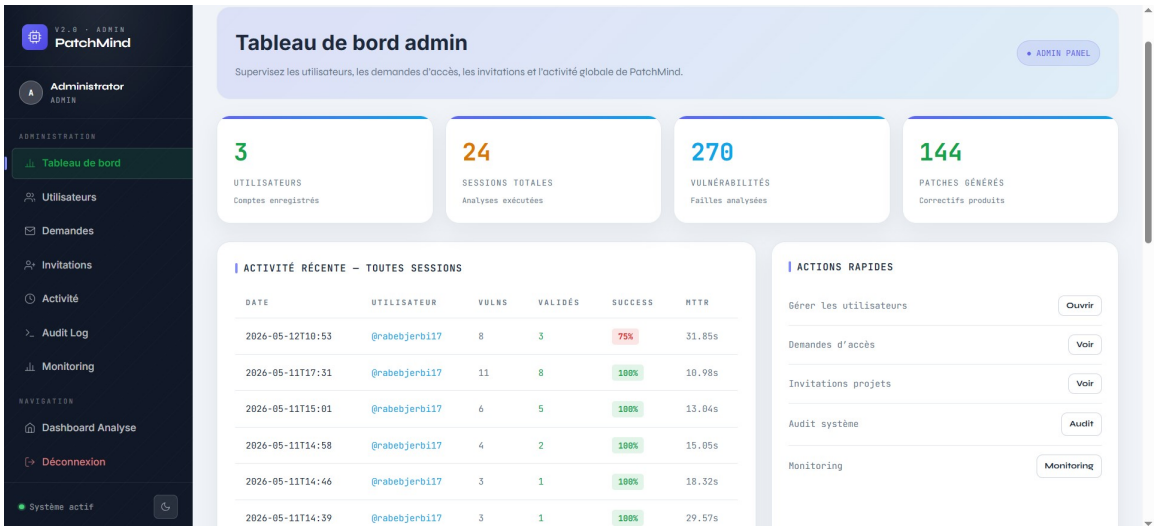


FIGURE 5.10 – Dashboard administrateur de PatchMind

5.3.2 Statistiques et métriques

Le dashboard intègre plusieurs indicateurs permettant d'évaluer l'activité et les performances globales de PatchMind.

Les métriques affichées comprennent :

— le nombre total d'analyses exécutées ;

— le nombre de vulnérabilités détectées ;

— la répartition des vulnérabilités par niveau de sévérité ;

— le nombre de correctifs générés ;

— le taux de réussite des validations ;

— le temps moyen d'analyse.

Ces indicateurs offrent aux administrateurs une vision synthétique de l'état de sécurité des projets analysés et permettent d'identifier les tendances et les axes d'amélioration.

La figure suivante présente l'interface de monitoring système incluant les métriques du consensus multi-LLM et du cache des correctifs :

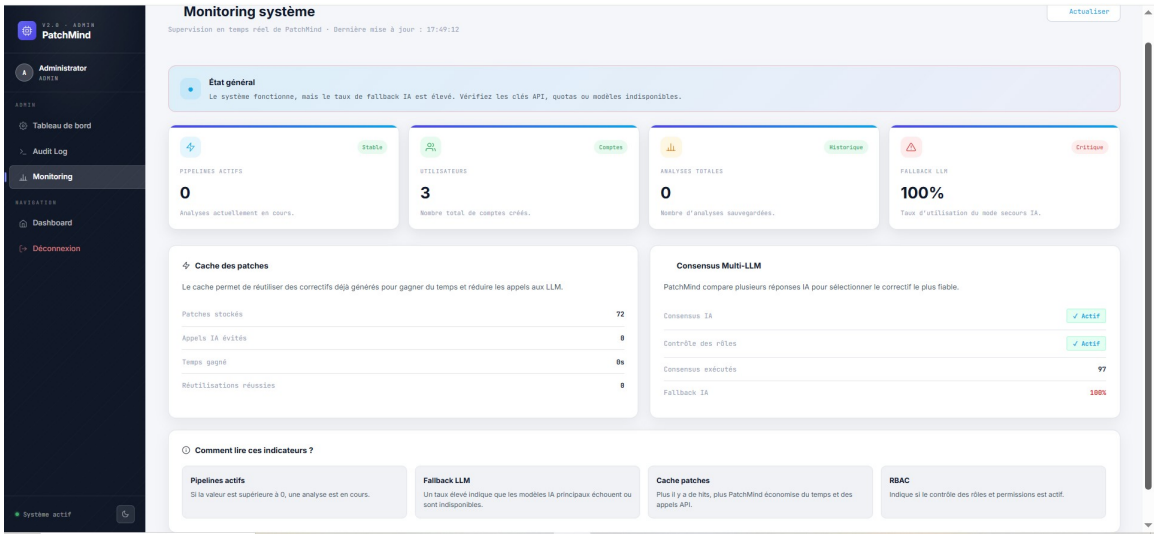


FIGURE 5.11 – Interface de monitoring et supervision du système PatchMind

5.3.3 Journaux d'audit

Afin d'assurer la traçabilité des actions réalisées sur la plateforme, un système de journaux d'audit a été implémenté. Chaque action importante est automatiquement enregistrée :

- connexions utilisateurs ;
- lancement des analyses ;
- génération des rapports ;
- actions administrateur ;
- modifications critiques.

Ces journaux permettent d'identifier certaines anomalies comportementales, d'améliorer la supervision de l'activité et de renforcer la sécurité globale de la plateforme.

La figure suivante présente l'interface des journaux d'audit de PatchMind, affichant le détail des actions avec horodatage, utilisateur et adresse IP :

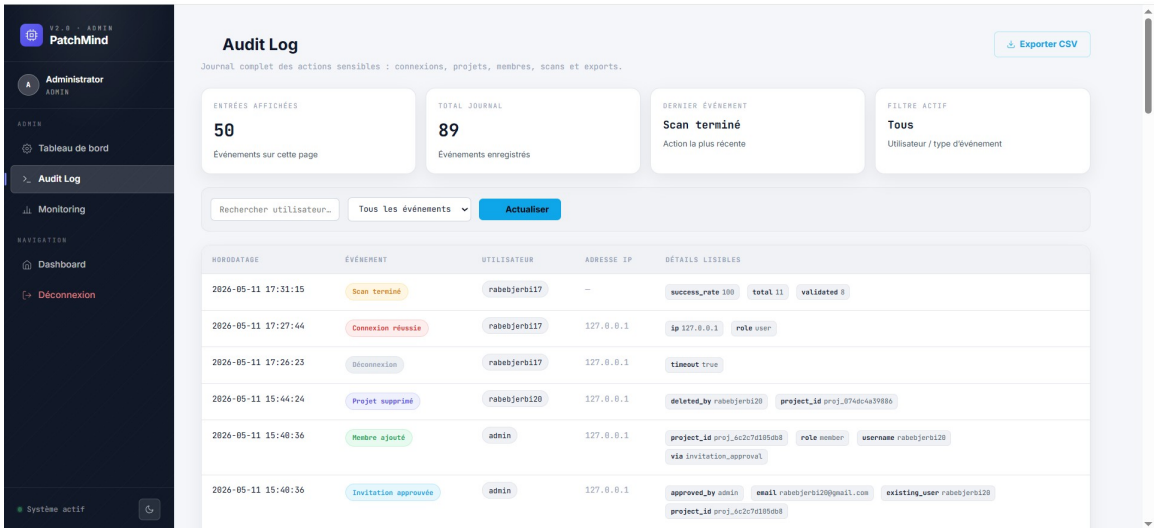


FIGURE 5.12 – Journaux d'audit de PatchMind

5.3.4 Tests unitaires

Des tests unitaires ont été réalisés à l'aide de Pytest afin de vérifier individuellement le bon fonctionnement des différents modules de la plateforme. La suite de tests est organisée en **16 fichiers spécialisés**, chacun dédié à un composant précis de la plateforme.

Les composants testés couvrent :

— le système d'authentification et les routes backend ;
— le contrôle d'accès basé sur les rôles (RBAC) ;
— la validation des correctifs et la détection de régressions ;
— le score de confiance des correctifs générés ;
— l'intégration des scanners de sécurité ;
— le stockage et la récupération des données ;
— les performances du pipeline d'analyse ;
— la sécurité de l'intégration GitHub Actions ;
— la soumission et le traitement des fichiers uploadés.

La figure suivante présente l'organisation de la suite de tests de PatchMind :

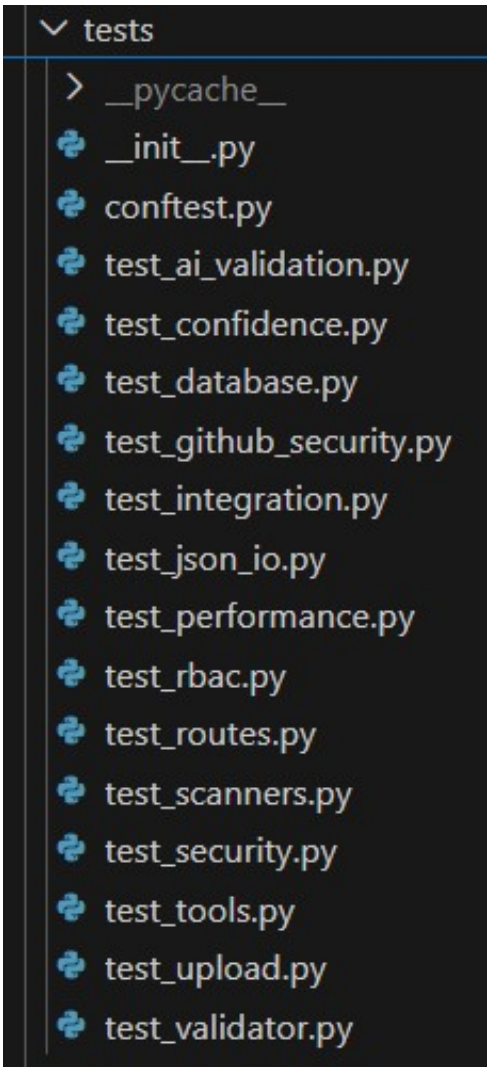


FIGURE 5.13 – Organisation de la suite de tests unitaires de PatchMind

Au total, **500 tests unitaires** ont été exécutés avec succès en **13.95 secondes**, couvrant l'ensemble des modules critiques de la plateforme. Chaque module a été testé indépendamment afin de détecter rapidement les erreurs de fonctionnement et de garantir la stabilité de chaque composant avant son intégration.

La figure suivante présente les résultats complets des tests unitaires exécutés avec Pytest :

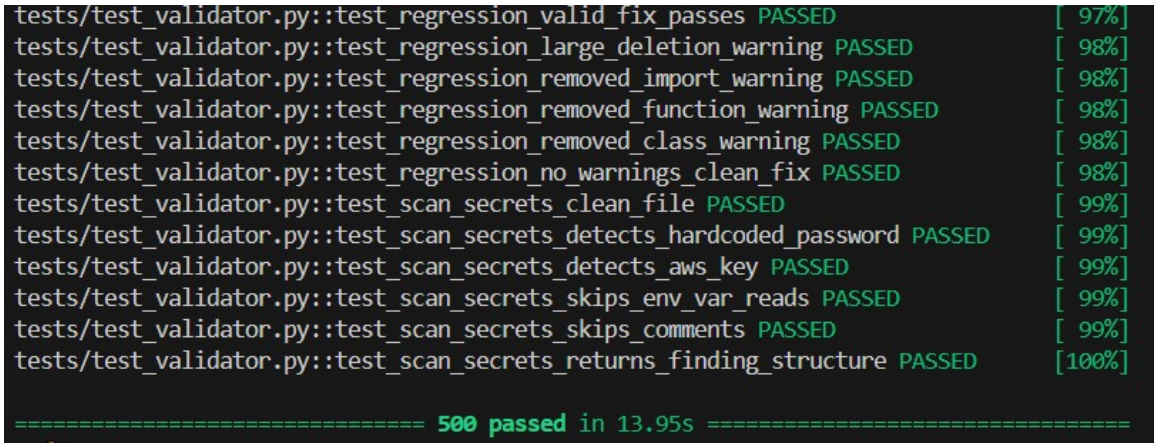


FIGURE 5.14 – Résultats des 500 tests unitaires Pytest de PatchMind

5.3.5 Tests d'intégration

Les tests d'intégration ont permis de vérifier la communication et la cohérence entre les différents modules de PatchMind.

Ces tests couvrent notamment :

— l'interaction entre le backend et les scanners de sécurité ;

— le stockage et la récupération des résultats ;

— la génération des rapports à partir des données d'analyse ;

— la consultation des analyses via l'interface utilisateur ;

— la communication avec les composants IA.

L'objectif était de s'assurer que l'ensemble des modules fonctionnent correctement de manière coordonnée et que les flux de données entre les composants sont fiables.

5.3.6 Tests de sécurité

Plusieurs tests de sécurité ont été réalisés afin d'évaluer la robustesse de la plateforme face aux menaces classiques des applications web.

Ces tests ont porté sur :

— la protection des sessions contre les attaques de type fixation et détournement ;

— le contrôle d'accès et la vérification que les routes protégées ne sont pas accessibles sans authentification ;

— la validation des entrées utilisateur pour prévenir les injections et les manipulations de paramètres ;

— la gestion des permissions entre les différents rôles ;

— la protection des données sensibles stockées en base.

Ces vérifications ont permis de corriger plusieurs failles potentielles et de renforcer la posture de sécurité globale de la plateforme.

5.3.7 Tests de performance

Des tests de performance ont été effectués afin d'évaluer les temps d'exécution des analyses et la réactivité globale de la plateforme.

Les éléments mesurés comprennent :

— le temps moyen de scan par outil ;
— le temps de génération des rapports PDF ;
— le temps de traitement et de normalisation des vulnérabilités ;
— la consommation des ressources système lors d'analyses simultanées.

Le tableau suivant présente les résultats moyens obtenus lors des tests de performance :

TABLE 5.1 – Résultats des tests de performance

Opération	**Temps moyen**
Scan Semgrep	6 secondes
Scan GitLeaks	4 secondes
Scan Snyk	9 secondes
Scan OWASP ZAP	25 secondes
Génération du correctif (RAG + LLM)	14.32 secondes
Génération du rapport PDF	3 secondes
Pipeline complet	392 secondes

Ces résultats ont permis d'identifier plusieurs optimisations, notamment la parallélisation partielle des scanners et la mise en cache des résultats intermédiaires, contribuant à réduire le temps global d'analyse.

5.4 Difficultés rencontrées

Durant cette release, plusieurs difficultés techniques ont été rencontrées, notamment au niveau de la génération IA, de la validation des correctifs et des performances globales de la plateforme.

5.4.1 Fiabilité des correctifs générés

Les modèles de langage peuvent produire des correctifs incomplets, syntaxiquement incorrects ou insuffisamment adaptés au contexte réel de la vulnérabilité. Pour réduire ce

risque, PatchMind a intégré une architecture RAG ainsi qu'un mécanisme de consensus multi-LLM afin d'améliorer la qualité des propositions générées.

La qualité des correctifs dépend fortement du contexte récupéré par l'architecture RAG ainsi que des capacités des modèles de langage utilisés. Certaines vulnérabilités complexes nécessitent encore une validation humaine avant déploiement en production.

5.4.2 Validation des correctifs

La validation automatique des correctifs a constitué un défi important. Il ne suffisait pas de générer un code corrigé ; il fallait également vérifier sa validité syntaxique, l'absence de nouvelles vulnérabilités et le risque de régression. Cette contrainte a conduit à la mise en place d'une validation multicouche.

5.4.3 Performances du moteur IA

La génération de correctifs et la recherche de contexte via FAISS peuvent augmenter le temps global d'analyse. Des optimisations ont donc été nécessaires afin de limiter les traitements redondants, améliorer la récupération des exemples similaires et réduire le temps de réponse global.

5.4.4 Cohérence entre reporting, historique et base de données

La synchronisation entre les rapports PDF, l'historique des analyses et les données stockées en base a nécessité une organisation rigoureuse des entités et des relations. L'objectif était de garantir que chaque rapport généré corresponde bien aux résultats stockés et consultables dans l'interface.

5.5 Résultats obtenus

Cette section présente les résultats obtenus à l'issue du développement complet de PatchMind, permettant d'évaluer les performances de la plateforme et l'atteinte des objectifs fixés.

5.5.1 Résultats de détection

Le pipeline multi-scanners de PatchMind a démontré sa capacité à détecter des vulnérabilités de nature variée sur des projets soumis lors des phases de test.

L'utilisation combinée de Semgrep, GitLeaks, Snyk et OWASP ZAP a permis d'obtenir une couverture plus large qu'une analyse basée sur un seul outil, en couvrant simultanément l'analyse statique, la détection de secrets, l'audit des dépendances et les tests dynamiques.

5.5.2 Qualité des correctifs générés

Le mécanisme de consensus multi-LLM a permis de produire des correctifs plus fiables que ceux issus d'un seul modèle de langage. Le score de confiance calculé pour chaque correctif constitue un indicateur permettant aux développeurs de prioriser les correctifs nécessitant une révision manuelle.

La validation automatique multicouche a également permis de rejeter les correctifs syntaxiquement invalides avant leur présentation à l'utilisateur, réduisant ainsi le risque de régression. Lors des tests réalisés, PatchMind a atteint un taux de réussite de **96.2%** avec un MTTR moyen de **14.32 secondes** et un score de confiance moyen de **72.2%**.

5.5.3 Apport de PatchMind par rapport aux approches classiques

Afin d'illustrer concrètement la valeur ajoutée de PatchMind, le tableau suivant compare le processus de remédiation classique avec l'approche automatisée proposée par la plateforme :

TABLE 5.2 – Comparaison entre l'approche classique et PatchMind

Processus	**Approche classique**	**Avec PatchMind**
Détection	Manuelle ou outil unique	Multi-scanners automatisés
Correction	Manuelle par le développeur	Générée automatiquement par IA
Validation	Revue manuelle	Validation automatique multicouche
Reporting	Manuel	Rapport PDF généré automatiquement
MTTR	Heures à plusieurs jours	14.32 secondes en moyenne

5.5.4 Performances de la plateforme

Les tests de performance ont confirmé que la plateforme est capable de traiter une analyse complète dans un délai raisonnable pour des projets de taille standard. Les optimisations apportées durant le Sprint 4, notamment la parallélisation partielle des scanners

et la mise en cache de certains résultats intermédiaires, ont contribué à réduire le temps global d'analyse par rapport aux premières implémentations séquentielles.

5.5.5 Réduction du MTTR

L'automatisation complète du cycle de traitement des vulnérabilités, depuis la détection jusqu'à la génération et la validation du correctif, contribue directement à la réduction du *Mean Time To Remediate* (MTTR).

Alors qu'une remédiation manuelle nécessite en moyenne plusieurs heures à plusieurs jours selon la complexité de la vulnérabilité, PatchMind génère et valide automatiquement un correctif en **14.32 secondes** en moyenne, contribuant ainsi à une réduction significative du MTTR et à une amélioration globale de la posture de sécurité des applications analysées.

5.6 Conclusion

L'ensemble des fonctionnalités développées durant cette deuxième release permet désormais à PatchMind de couvrir l'intégralité du cycle DevSecOps, depuis l'analyse de sécurité jusqu'à la génération et la validation automatique des correctifs.

Le troisième sprint a apporté la génération intelligente de correctifs par consensus multi-LLM, la validation automatique multicouche, la production de rapports PDF structurés et la gestion de l'historique des analyses. Le quatrième sprint a permis de développer le dashboard administrateur, les journaux d'audit et une couverture de tests complète couvrant les aspects fonctionnels, d'intégration, de sécurité et de performance.

Les résultats obtenus confirment que PatchMind atteint ses objectifs principaux en matière de détection, de remédiation automatisée et de réduction du MTTR. PatchMind illustre ainsi le potentiel des technologies d'intelligence artificielle générative dans l'automatisation des processus DevSecOps et de remédiation applicative.

Le chapitre suivant présente la conclusion générale du projet, dresse un bilan des objectifs atteints et expose les perspectives d'amélioration et d'évolution envisagées pour PatchMind.

Conclusion Générale

L'évolution rapide des environnements numériques et la multiplication des cybermenaces rendent aujourd'hui la sécurisation des applications plus complexe que jamais. La croissance des bases de code, l'utilisation massive des applications web ainsi que l'intégration continue des services cloud augmentent considérablement la surface d'attaque des systèmes modernes. Dans ce contexte, les vulnérabilités applicatives représentent un enjeu majeur pour les organisations, d'autant plus que les délais de remédiation demeurent souvent élevés.

C'est dans cette optique qu'a été développé **PatchMind**, une plateforme DevSecOps intelligente dédiée à la détection, à l'analyse et à la remédiation automatisée des vulnérabilités applicatives. L'objectif principal du projet était de concevoir une solution centralisée capable d'intégrer plusieurs outils de sécurité, d'exploiter l'intelligence artificielle pour générer automatiquement des correctifs et de réduire significativement le *Mean Time To Remediate* (MTTR).

Le projet a permis d'intégrer plusieurs scanners complémentaires — *Semgrep*, *GitLeaks*, *Snyk/OSV* et *OWASP ZAP* — afin de couvrir différents aspects de la sécurité applicative, notamment l'analyse statique, l'analyse dynamique, l'audit des dépendances ainsi que la détection de secrets exposés. Cette approche multi-scanners garantit une couverture plus complète des analyses et améliore la capacité de détection des vulnérabilités au sein d'un pipeline unifié.

PatchMind exploite également une architecture basée sur le *Retrieval-Augmented Generation* (RAG) associée à un mécanisme de consensus multi-LLM afin d'améliorer la pertinence et la fiabilité des correctifs générés automatiquement. Le système s'appuie sur une base vectorielle FAISS permettant de rechercher des contextes similaires et d'enrichir les requêtes transmises aux modèles de langage. Plusieurs mécanismes de validation ont ensuite été mis en place afin de vérifier la validité syntaxique des correctifs, de détecter les éventuelles régressions et de confirmer la suppression effective des vulnérabilités détectées.

Au-delà des mécanismes d'analyse et de remédiation, le projet a permis de développer une plateforme complète intégrant l'authentification sécurisée avec double authentification (2FA), le contrôle d'accès basé sur les rôles (RBAC), la gestion des demandes d'accès, les

journaux d'audit, la génération automatique de rapports PDF, la gestion collaborative des projets, un pipeline d'intégration continue GitHub Actions automatisant les contrôles de sécurité à chaque commit, ainsi qu'un dashboard administrateur centralisant les métriques et statistiques de supervision.

Les différentes phases de tests réalisées ont permis de valider la stabilité, la sécurité et les performances globales du système. La suite de tests mise en place, composée de **500 tests unitaires** répartis en 16 modules spécialisés, a permis de valider la robustesse de l'ensemble des composants de la plateforme. Les résultats obtenus montrent que la plateforme atteint un taux de réussite de **96.2%** avec un MTTR moyen de **14.32 secondes**, confirmant que l'automatisation du cycle complet de traitement des vulnérabilités — depuis la détection jusqu'à la validation des correctifs — permet de réduire significativement le délai de remédiation tout en améliorant la centralisation, l'automatisation et l'efficacité des processus de sécurisation applicative.

Ce projet a également constitué une expérience particulièrement enrichissante sur les plans technique et méthodologique. Il a permis d'approfondir plusieurs domaines tels que le développement web sécurisé, les architectures DevSecOps, l'intégration des outils de cybersécurité, les bases de données relationnelles, les pipelines automatisés, les systèmes collaboratifs ainsi que l'exploitation de l'intelligence artificielle appliquée à la sécurité logicielle.

Plusieurs perspectives d'amélioration peuvent être envisagées afin d'enrichir davantage la plateforme. Sur le plan de la détection, l'intégration de nouveaux scanners et le support de langages de programmation supplémentaires permettraient d'élargir la couverture des analyses. Sur le plan de l'intelligence artificielle, l'adoption de modèles plus avancés et l'amélioration continue du moteur RAG contribueraient à produire des correctifs encore plus précis et mieux adaptés au contexte des vulnérabilités. Sur le plan de l'infrastructure, la migration vers une architecture distribuée ainsi que le déploiement dans un environnement cloud scalable permettraient d'améliorer les performances et la disponibilité de la plateforme pour des volumes d'analyses plus importants. Enfin, l'intégration complète du moteur de remédiation dans des pipelines CI/CD représenterait une évolution naturelle vers une automatisation avancée des processus DevSecOps.

Ainsi, PatchMind représente une contribution concrète à l'automatisation intelligente de la sécurité applicative et illustre le potentiel des approches combinant DevSecOps et intelligence artificielle dans l'amélioration des processus de détection, de remédiation et de supervision des vulnérabilités logicielles.